Assignment 1: Reinforcement Learning

Jianong Xu (GitHub: EvelynXuNU)

October 29, 2026

Repository: **iems_HW1**

Source and experiment evidence: https://github.com/EvelynXuNU/iems_HW1

The required experiments use dynamic programming for shuttle dispatching and PyTorch REINFORCE with and without a lagged scalar baseline. Every technical part includes its assumptions, labeled plots, quantitative validation, edge cases, execution directions, and proposed experiments for three additional months.

Results at a glance. The shuttle's infinite-horizon optimum dispatches at 13 or more passengers, with expected discounted cost $J^*(0) = 576.670168$. The policy results below each use 500 frozen, stochastic evaluation episodes; parentheses contain sample standard deviations.

Experiment	CartPole score	Pong score
Part 1: REINFORCE, no baseline	181.612 (97.2304)	-19.046 (2.5987)
Part 2: REINFORCE, lagged EMA	438.342 (95.7679)	-19.460 (0.9952)
Additional CNN / EMA pilot	Not run	-18.492 (1.8722)
Additional single-GPU PPO	500.000 (0.0000)	-6.626 (4.9727)

The two required Pong runs and the CNN pilot had no wins. Fresh single-GPU PPO training and evaluation produced 45/500 Pong wins and 500/500 CartPole games at the cap. PPO is supplementary because it changes the algorithm and Pong representation. These results are conditional on one training seed; Pong is not solved and CartPole's observed zero SD is not population certainty.

Execution overview. Use Python 3.11 or newer. From the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-q2.txt
```

The shuttle can also run with the smaller requirements.txt dependency set. The execution subsections provide training, evaluation, and validation commands. Fresh output directories preserve the recorded experiments. CUDA commands use exactly one assigned GPU; CPU validation is sufficient for the mathematical checks. The optional PPO study has separate dependencies in requirements-ppo.txt.

Contents

Value and Policy Iteration: Shuttle Dispatching **3**

MDP and explicit assumptions . . . 3

(a) Enumeration for 500 periods . . . 3

(b) Infinite-horizon value iteration . . . 4

(c) Policy iteration . . . 5

Edge cases and the evidence used to check them . . . 6

Validation experiments and reproducibility . . . 7

What I would test with three additional months . . . 11

Policy Gradient: Part 1 **12**

Objective and gradient . . . 12

Environments and policies . . . 12

CartPole results and required plots . . . 13

Pong results and required plots . . . 15

Edge cases and quantitative validation . . . 17

Execution and reproducibility . . . 18

With three additional months . . . 19

Policy Gradient: Part 2 **20**

Baseline and implementation . . . 20

CartPole results . . . 21

Gradient-variance experiments . . . 23

Edge cases and further validation . . . 24

Pong reference result and runtime . . . 26

Execution and reproducibility . . . 27

Budgeted Pong follow-up . . . 28

With three additional months . . . 29

Supplement: Single-GPU PPO Performance Study **30**

Single-GPU execution and method . . . 30

Frozen-policy results . . . 30

Interpretation and reproduction . . . 33

Value and Policy Iteration: Shuttle Dispatching

All results below use the specified parameters $K = 15$, $c_f = 100$, $c_h = 2$, $\gamma = 0.95$, and uniform integer arrivals on $\{1, 2, 3, 4, 5\}$. Computed values are **expected discounted costs**; smaller is better. In a reward-maximization convention, define $r = -c$ and $V = -J$.

MDP and explicit assumptions

The state $s_t \in \{0, 1, \dots, 200\}$ is the number of passengers waiting immediately before the dispatch decision. Action $a_t = 0$ waits and $a_t = 1$ dispatches one shuttle. A partially filled shuttle is allowed. Dispatch removes $\min(s_t, K)$ passengers, so the queue immediately after service is

$$q(s, a) = \max(s - aK, 0).$$

Each period follows this order: observe the queue, decide whether to dispatch, remove passengers, charge the dispatch and holding costs, and then add arrivals. Thus new arrivals join the next decision epoch and do not incur the current period's holding cost. The one-period cost is

$$c(s, a) = c_f a + c_h q(s, a) = 100a + 2\max(s - 15a, 0).$$

Arrivals are assumed independent and identically distributed across periods, independent of decisions. The transition is

$$s_{t+1} = \min\{200, q(s_t, a_t) + A_t\}, \qquad \Pr(A_t = k) = \tfrac{1}{5},\ k = 1, \dots, 5.$$

Equivalently,

$$P(s' \mid s, a) = \frac{1}{5} \sum_{k=1}^{5} \mathbf{1}\{s' = \min(200, q(s, a) + k)\}.$$

The queue limit is modeled as a hard station capacity: excess arrivals are turned away without an additional penalty or outside backlog. The assignment does not specify an overflow rule, so this is an explicit modeling assumption. A new shuttle is available at each epoch; vehicle availability and travel time impose no additional constraints. These assumptions make queue length a sufficient Markov state.

For the finite-horizon problem, $T = 500$ means decisions at times $0, \dots, 499$, followed by terminal cost $J_{500}(s) = 0$. No forced final clearing or penalty for passengers remaining at termination is added. Exact action ties select wait. The infinite-horizon objective is

$$J^*(s) = \min_{\pi} \mathbb{E}_s^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t c(s_t, a_t) \right].$$

(a) Enumeration for 500 periods

For the 500-period problem, I used backward induction, evaluating both actions and all five arrival outcomes at every state and time. This preserves the ability to adapt future actions to observed arrivals. Enumerating only predetermined sequences of actions would instead solve an open-loop problem. Expanding every full decision tree for 500 periods would be intractable; backward induction reuses identical state-time subproblems.

Starting with $J_{500}(s) = 0$, calculate, for $t = 499, \dots, 0$,

$$Q_t(s, a) = 100a + 2q(s, a) + \frac{0.95}{5} \sum_{k=1}^{5} J_{t+1}(\min(200, q(s, a) + k)),$$

$$J_t(s) = \min_{a\in\{0,1\}} Q_t(s,a), \qquad \pi_t(s) \in \arg\min_a Q_t(s,a).$$

This requires $500 \cdot 201 \cdot 2 \cdot 5 = 1{,}005{,}000$ action-outcome evaluations. The implementation saves the value and policy at every time.

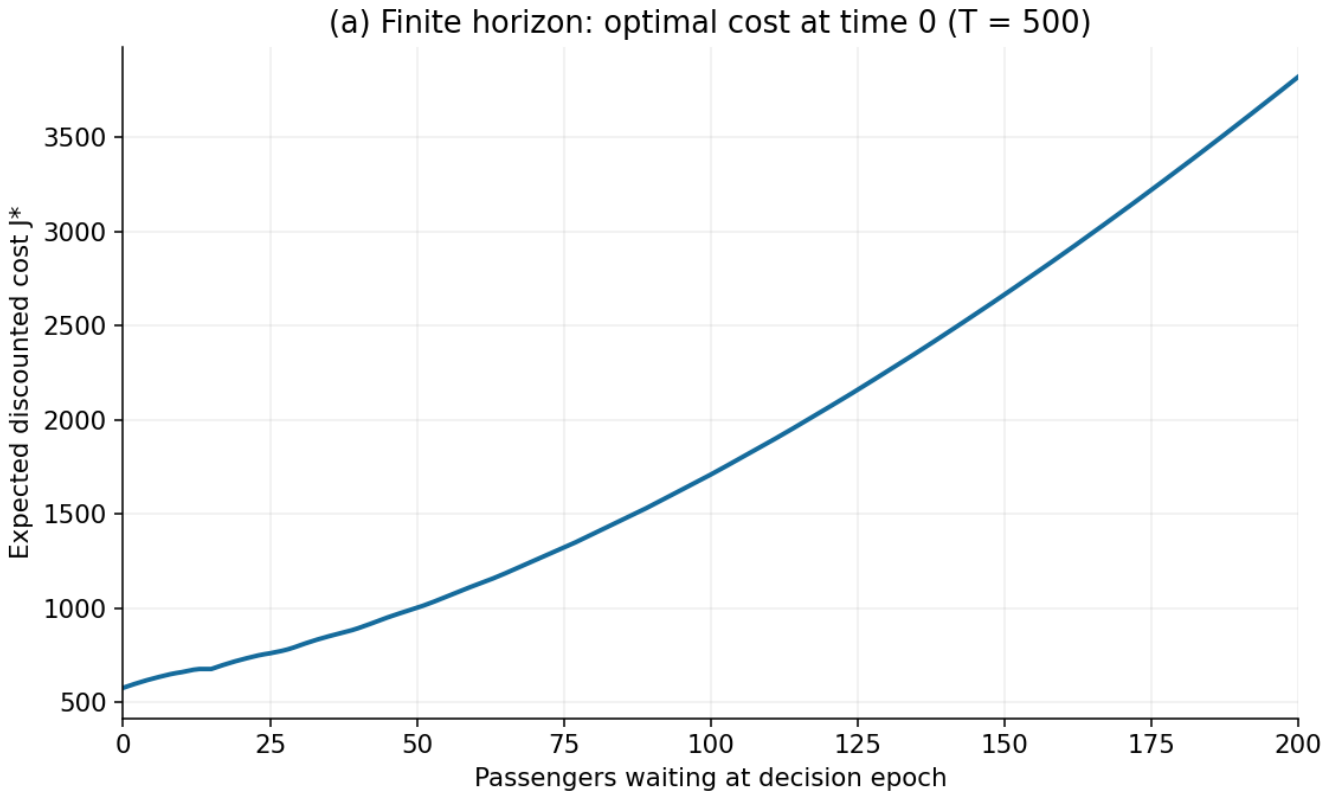


Figure 1. Optimal discounted cost at time 0 for every initial queue size, with 500 remaining decisions and zero terminal cost.

(b) Infinite-horizon value iteration

The Bellman operator is

$$(\mathcal{T}J)(s) = \min_{a\in\{0,1\}} \left\{ 100a + 2q(s,a) + \frac{0.95}{5} \sum_{k=1}^{5} J(\min(200, q(s,a)+k)) \right\}.$$

Initialize $J^{(0)} = 0$ and synchronously update $J^{(n+1)} = \mathcal{T}J^{(n)}$. I stop when the Bellman residual of the **returned** value satisfies

$$\|\mathcal{T}J - J\|_\infty \le (1-\gamma)10^{-8} = 5 \times 10^{-10}.$$

The discounted Bellman operator is a γ-contraction: averaging cannot increase the maximum difference between two value vectors, multiplication by γ shrinks it, and taking a minimum preserves this bound. Therefore

$$\|J - J^*\|_\infty \le \frac{\|\mathcal{T}J - J\|_\infty}{1-\gamma}.$$

Convergence took **487 updates**. The final residual was 4.8203×10^{-10}, giving a value-error bound of 9.6406×10^{-9}, apart from floating-point rounding.

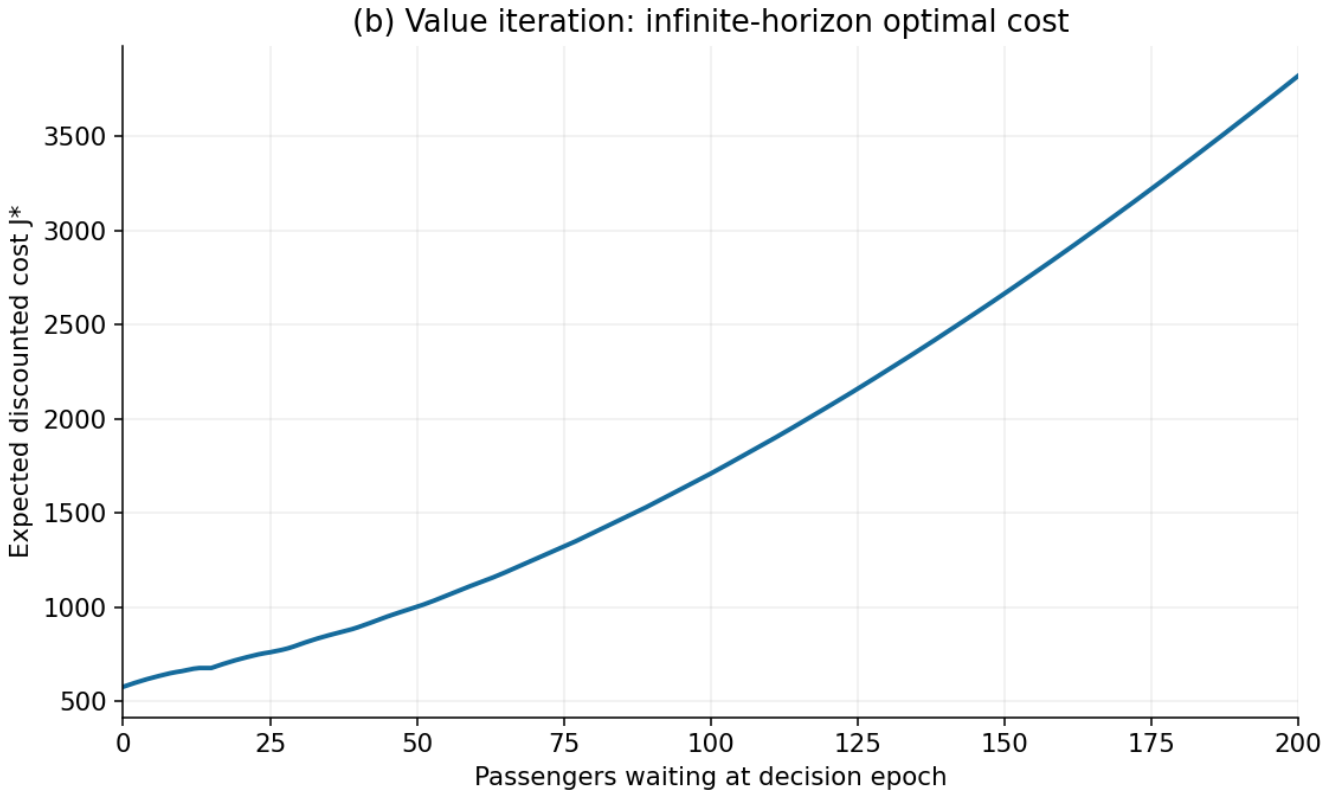


Figure 2. Infinite-horizon optimal discounted cost from value iteration. Figures 1 and 2 look identical because their maximum numerical difference is negligible at this plotting scale.

(c) Policy iteration

Start with the all-wait policy. For a fixed policy π, form its transition matrix P_π and cost vector c_π, and solve

$$(I - 0.95P_\pi)J_\pi = c_\pi.$$

This system is invertible because $\gamma < 1$ and P_π is stochastic. Improve the action at every state using the Bellman action values evaluated at J_π, and repeat until the policy is unchanged. All 201 states are optimized without imposing a threshold structure.

The algorithm used **6 policy evaluations**; the numbers of changed actions after those evaluations were 196, 28, 22, 7, 1, and 0. The resulting policy is

$$\boxed{\pi^*(s) = \begin{cases} 0 & 0 \le s \le 12, \\ 1 & 13 \le s \le 200. \end{cases}}$$

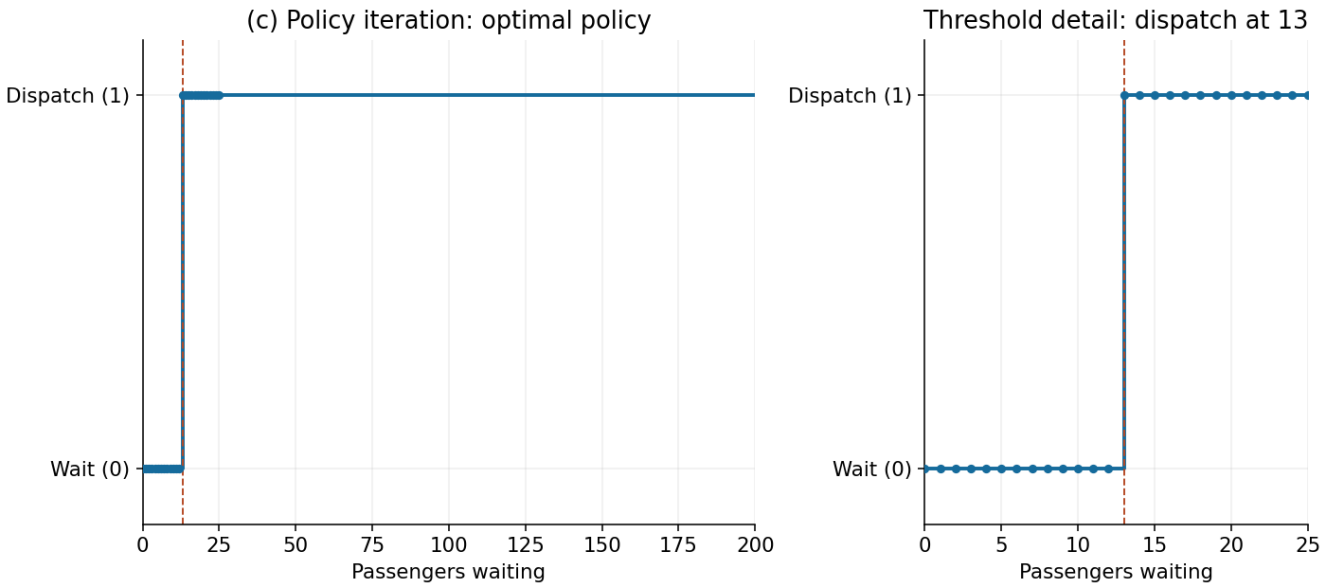


Figure 3. Optimal policy over the full state space, with a close-up of the dispatch boundary. Queue lengths are integers; connecting steps are visual aids.

The optimal policy dispatches before the shuttle is full because the objective balances the fixed dispatch cost against current and future waiting costs. Let $\Delta(s) = Q(s,0) - Q(s,1)$; a positive difference favors dispatch. At 12 passengers, waiting is cheaper by 3.778004; at 13, dispatching is cheaper by 4.099663. The smallest absolute action gap over all states is 3.778004. These gaps are much larger than the numerical error.

Initial passengers	Finite-horizon cost at time 0	Infinite-horizon cost	Infinite-horizon action
0	576.670168	576.670168	Wait
12	672.892163	672.892163	Wait
13	676.670168	676.670168	Dispatch
15	676.670168	676.670168	Dispatch
50	1001.527732	1001.527732	Dispatch
100	1709.042541	1709.042541	Dispatch
200	3821.586477	3821.586477	Dispatch

The plateau at states 13, 14, and 15 is expected: dispatch empties the station, costs 100, and gives the same next-state distribution. In particular, $J^*(13) = J^*(14) = J^*(15) = 100 + J^*(0)$.

Edge cases and the evidence used to check them

Edge case	Expected behavior and handling	Quantitative check
Empty station	Dispatch serves nobody and adds 100; wait is optimal.	$Q(0,0) - Q(0,1) = -100$.
Partial shuttle and exact capacity	Serve $\min(s, 15)$, allow empty seats, and never create negative queues.	States 13, 14, and 15 all have cost 676.670168 and dispatch.
Queue above shuttle capacity	A single action serves at most 15, rather than clearing the station.	At state 200, dispatch leaves 185, costs $100 + 2(185) = 470$, and the next state is 186 through 190.
Upper boundary	Aggregate all arrival probability that maps to the cap.	Waiting at 198 leads to 199 with probability 0.2 and to 200 with probability 0.8; waiting at 200 stays at 200 with probability 1.
Finite-horizon termination	Use zero terminal value and allow time-dependent decisions.	At time 499, wait in all 201 states: dispatch saves at most 30 holding cost but costs 100.

Edge case	Expected behavior and handling	Quantitative check
Numerical convergence and ties	Check Bellman residual on the returned vector; choose wait on exact ties.	Baseline minimum action gap is 3.778004; VI and PI agree at all 201 states. Free dispatch at an empty station creates a genuine tie and selects wait.
No arrivals	Discounting and absence of a service deadline may make indefinite waiting optimal for a tiny queue.	Exact costs for states 0, 1, 2, 3 are 0, 40, 80, 100; wait at 1 and 2 and dispatch at 3.
Degenerate economic regimes	Zero holding cost favors waiting; free service favors dispatch; zero discount removes all future benefits.	These analytical predictions pass across the state space; details below.
Policy shape under changed parameters	Discover each action rather than requiring a threshold in advance.	With $K = 5$, dispatch at 5 through 192, then wait again at 193 through 200 under the free-overflow assumption.
Deterministic simulation returns	Separate omitted future cost from sampling uncertainty.	Always-dispatch has zero sampling SE; its 400-step mean matches the truncated analytic value within 2.3×10^{-12}.

The queue cap deserves particular attention. Under the assigned optimum, waiting occurs only at states at most 12, so a wait transition reaches at most 17. Dispatching from any state at most 200 reaches at most 190. Thus the optimal chain never clips arrivals. Expanding the cap to 300 gives identical actions on states 0 through 200 and changes their costs by at most 4.55×10^{-13}.

That finding is specific to the assigned parameters. When dispatch costs 10^6, both caps lead to all-wait policies; increasing the cap from 200 to 300 increases the cost from initial state 200 by 1863.445962. Similarly, the $K = 5$ example shows how free rejected demand can make a full station economically attractive. This is a consequence of the assumed objective and overflow rule; it motivates sensitivity analysis rather than assuming every variant must have one dispatch threshold.

Validation experiments and reproducibility

Use Python 3.10 or newer. From the project directory, create an environment and install the dependencies, then generate the results and run the validation:

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
python3 q1_shuttle.py
python3 validate_q1.py
python3 verify_q1_baseline.py
python3 verify_q1_exhaustive.py
```

`q1_shuttle.py` generates the figures and saved values under `results/q1/`. `validate_q1.py` runs the ten experiments below and raises an assertion error if a prediction fails. The evidence is saved in `results/q1/validation/validation.json` and the named CSV files. I checked the implementation against a small decision tree, closed-form cases, parameter changes, and an independent simulator. The final two commands run separate verifiers, described after those ten experiments.

1. **Bellman optimality and agreement.** Policy iteration's value has maximum Bellman residual 4.55×10^{-13}, equivalent to an approximate value-error bound 9.09×10^{-12}. VI and PI differ by at most 9.62×10^{-9} and have zero action disagreements. The PI value is nondecreasing over the queue states, within rounding. A residual checks the Bellman fixed point directly, instead of relying only on two solvers agreeing.

2. **Independent literal decision-tree enumeration.** A separate scalar routine expands every action and arrival branch without using the main Bellman or transition routines. Test a seven-state model with $K = 3, c_f = 4, c_h = 1$, $\gamma = 0.9$, and arrivals 0, 1, 3 with probabilities 0.2, 0.3, 0.5. Across every starting state and horizons 0 through 4, all **35 comparisons** with backward induction agree exactly. The tiny problem exercises partial service, the cap, nonuniform expectations, and termination.

3. **No-arrival closed-form solution.** When $A_t = 0$, evaluate all choices of sending m consecutive shuttles and then waiting forever:

$$\min_{0 \le m \le \lceil s/K \rceil} \left\{ \sum_{j=0}^{m-1} \gamma^j [c_f + c_h \max(s - (j+1)K, 0)] + \gamma^m \frac{c_h \max(s - mK, 0)}{1 - \gamma} \right\}.$$

This covers an optimal policy: at an unchanged state with no arrivals, waiting either continues forever or can be replaced by immediate dispatch when dispatch is the better option. The formula agrees with PI over all 201 states within 1.82×10^{-12}. For one passenger, waiting forever costs $2/(1 - 0.95) = 40 < 100$, an easily checked consequence of discounting.

4. **Deterministic arrivals and a transparent trajectory.** Replace arrivals by exactly three passengers. From an empty station the optimal path is $0, 3, 6, 9, 12, 15, 3, \ldots$. After the initial zero-cost wait, the five-step cycle has costs 6, 12, 18, 24, 100. Therefore

$$J(0) = \frac{\gamma(6 + 12\gamma + 18\gamma^2 + 24\gamma^3 + 100\gamma^4)}{1 - \gamma^5} = 569.753018714.$$

PI agrees within 1.14×10^{-12}. The first ten transitions and the identity $J(s_t) = c_t + \gamma J(s_{t+1})$ are checked and saved in `deterministic_first_10_steps.csv`. This also shows that replacing random arrivals by their mean would change the original problem: its cost at zero is 576.670168, rather than 569.753019.

Time	Queue	Action	Post-service queue	Cost	Arrival	Next queue
0	0	Wait	0	0	3	3
1	3	Wait	3	6	3	6
2	6	Wait	6	12	3	9
3	9	Wait	9	18	3	12
4	12	Wait	12	24	3	15
5	15	Dispatch	0	100	3	3
6	3	Wait	3	6	3	6
7	6	Wait	6	12	3	9
8	9	Wait	9	18	3	12
9	12	Wait	12	24	3	15

5. **Analytical economic limits.** With $c_h = 0$, all-wait has exactly zero cost. With $c_f = 0$, dispatch at every positive queue, and the optimal cost is zero on states 0 through 15, since subsequent arrivals never exceed 5. With $\gamma = 0$, all-wait is optimal because a dispatch costs more than its maximum immediate saving. With $K = 0, A_t = 0$, dispatch cannot help and $J(s) = 2s/(1 - 0.95)$. The largest absolute discrepancy among these analytic checks is below 2.0×10^{-12}.

6. **Horizon convergence and final-period behavior.** Test horizons 0, 1, 5, 20, 100, and 500. Values increase toward the infinite-horizon cost. Since every state-action cost is at most 470,

$$0 \le J^*(s) - J_0^{(T)}(s) \le \frac{470\gamma^T}{1 - \gamma}.$$

At $T = 500$, the maximum observed difference is 4.94×10^{-9}, below the bound 6.84×10^{-8}; the time-0 policies agree at every state. The last-period policy is all-wait. Full results are in `horizon_convergence.csv`.

Limitation of this comparison: VI initialized at zero after n updates is algebraically the n-period finite-horizon value. Agreement between these two calculations is a consistency check, not independent proof; the scalar tree, closed forms, and simulation provide additional evidence.

7. **Queue-cap sensitivity.** Check the hand-calculated boundary probabilities and compare caps 200 and 300. Also test expensive dispatch to demonstrate a regime in which the overflow assumption materially changes the result. Numerical outcomes are discussed above.

8. **Economic sensitivity.** Vary dispatch cost over 50, 100, 200; holding cost over 1, 2, 4; capacity over 5, 15, 30; and discount over 0.5, 0.8, 0.95. Verify the direction of changes in optimal cost at **every** state. Higher dispatch or holding cost cannot reduce the optimum; greater capacity cannot raise it; increasing the discount factor cannot lower the discounted cost with nonnegative costs. All 12 settings pass. As illustrative behavioral checks, dispatch-cost increases shift the threshold from 11 to 13 to 14; holding-cost increases shift it from 14 to 13 to 11. The smaller-capacity case is deliberately allowed to have a nonthreshold policy. See `parameter_sensitivity.csv`.

9. **Competing-policy comparisons.** Solve the linear systems for always-wait, always-dispatch, and dispatch-when-full policies. Check optimal-policy dominance from all 201 initial states. From an empty station:

Policy	Exact infinite-horizon discounted cost
Optimal, dispatch at 13	576.670168
Dispatch when full, at 15	597.355975
Always dispatch	2000.000000
Always wait	2203.913256

   The optimal policy improves on waiting for a full shuttle by approximately **3.46%**. Always-dispatch has the independent formula $100/(1-0.95) = 2000$: every arrival batch fits in a shuttle, and a dispatch is paid even at the empty initial state.

10. **Independent Monte Carlo evaluation.** The simulator directly updates passengers, costs, and arrivals without calling the production Bellman or transition routines. Use **20,000 trajectories of 400 periods**, seed 3817, for the optimum from initial states 0, 12, 13, 200 and each competing policy from state 0. Common random arrivals aid comparison. Compare sample means against exact fixed-policy **400-period** expectations; report approximate 95% intervals as mean ± 1.96 standard errors. The remaining infinite tail is bounded separately by 1.155×10^{-5}.

Policy	Initial state	Simulated mean	Approximate 95% interval	Exact 400-period cost
Optimal	0	576.5852	[576.1485, 577.0219]	576.6702
Optimal	12	672.6872	[672.2616, 673.1128]	672.8922
Optimal	13	676.5852	[676.1485, 677.0219]	676.6702
Optimal	200	3820.7493	[3819.7004, 3821.7983]	3821.5865
Always wait	0	2202.8474	[2200.6036, 2205.0912]	2203.9132
Always dispatch	0	2000.0000	Deterministic	2000.0000
Dispatch at 15	0	597.3140	[596.8500, 597.7779]	597.3560

   All stochastic cases are within 1.565 standard errors of their exact truncated expectations. The diagnostic assertion permits five standard errors to avoid using a nominal 95% CI as a certainty test. The deterministic case agrees within rounding. There are zero overflow steps in all optimal trajectories, including those starting at 200. See `simulation_comparison.csv` for unrounded results. These intervals quantify simulation sampling error; they do not quantify uncertainty about the model.

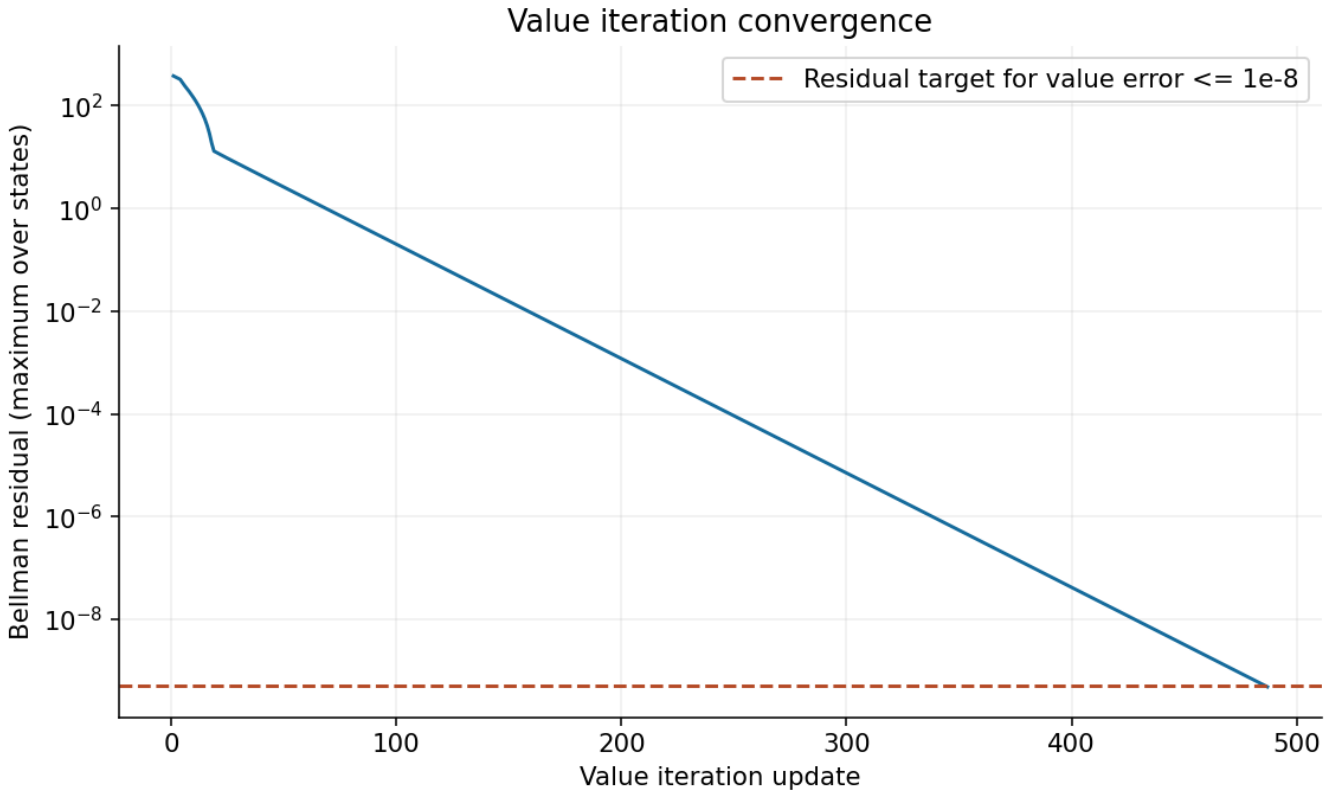


Figure 4. Maximum Bellman residual across states after each value-iteration update. The dashed line is the stopping target for a value error of at most 10^{-8}.

Additional independent verification

`verify_q1_baseline.py` imports neither the solver nor the validation module. It reconstructs every finite-horizon calculation using scalar loops, assembles the infinite-horizon policy's linear equations independently, and checks all 202 dispatch thresholds, including always dispatch and never dispatch.

Check	Observed result
All 100,701 finite-horizon values, including the terminal row	Maximum difference 3.18×10^{-12}.
All 100,500 finite-horizon actions	Zero disagreements.
Independently reconstructed Bellman equation	Residual 9.09×10^{-13}, corresponding to a numerical value-error bound 1.82×10^{-11}.
All 202 threshold policies, evaluated from every state	None improves on the claimed optimum, within numerical tolerance; threshold 13 is best from an empty station.

Threshold enumeration tests only that class of policies. The independently reconstructed Bellman optimality equation also checks both actions at every state, establishing optimality for the full discounted MDP to numerical precision. The threshold-12, threshold-13, threshold-14, and threshold-15 costs from an empty station are 581.664264, 576.670168, 581.581291, and 597.355975, respectively. Evidence is saved in `results/q1/verifier/independent_audit.json`.

`verify_q1_exhaustive.py` uses a different optimization method: it evaluates **every deterministic stationary policy** on each of 30 six-state models. Its oracle builds transitions and costs independently of the production model's Bellman and transition methods. Each model has $2^6 = 64$ policies, giving **1,920 policies** in total. For finite discounted MDPs an optimal deterministic stationary policy exists, so this enumeration supplies a complete optimum for each tiny problem.

The cases include zero holding cost, free dispatch, zero capacity, zero discount, no arrivals, repeated and nonuniform arrival outcomes, arrivals above the queue limit, excess shuttle capacity, and 24 random models with seed 82431.

The maximum PI value error against the oracle is 1.19×10^{-12}; the maximum VI value error is 9.975×10^{-9}, below its 10^{-8} target. Both solvers select actions with zero observed suboptimality against the oracle. The check accepts either action when both are optimal, rather than requiring identical action labels in tied cases. These checks cover the assigned problem and selected variants, not every possible parameter choice. Evidence is saved in `results/q1/verifier/exhaustive.json`.

What I would test with three additional months

Month 1: broader numerical certification. Create an independent discounted MDP linear-programming solution, extend the current exhaustive checks to larger small-state models, and use higher-precision arithmetic around nearly tied actions. Sweep costs, capacities, queue limits, and discount factors approaching one; record Bellman residuals, policy gaps, convergence rates, and disagreement locations. This would test both numerical stability and whether apparent policy structure persists beyond the assigned example.

Month 2: robustness to modeling assumptions. Compare rejected arrivals, penalized overflow, an explicitly modeled outside backlog, and larger queue limits. Test bursty, nonuniform, and correlated arrival processes with equal means to separate effects of demand level and variability. For correlation, augment the state with the demand regime so the model remains Markov. Compare arrival/service timing, terminal penalties, and required final clearing. Report the cost of applying the original policy under each changed environment and the benefit of reoptimizing. The nonthreshold $K = 5$ result makes overflow modeling a particularly useful target.

Month 3: service performance and out-of-sample evaluation. Run longer simulations under multiple independent seeds, including sudden demand shifts and queues near capacity. Track passenger waiting times, tail waiting probabilities, rejected demand, occupancy, dispatch frequency, and regret relative to a reoptimized policy, with uncertainty intervals. Add minimum-service or maximum-wait constraints and examine the cost-service tradeoff. If real arrival data are available, fit demand models on one time interval and evaluate on a separate interval. This would assess practical service quality as well as the discounted cost objective.

Policy Gradient: Part 1

GitHub username: **EvelynXuNU**. Repository: **iems_HW1**. Source and experiment evidence: https://github.com/EvelynXuNU/iems_HW1.

The implementation is PyTorch episodic REINFORCE with Adam. Both CartPole-v1 and Pong-v5 training and their required 500-game evaluations are complete. It has **no baseline, return centering, advantage normalization, critic, entropy bonus, reward clipping, or point-boundary resets**. Part 2 remains a separate experiment.

Objective and gradient

For one episode of length T, the objective is

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right].$$

Define the reward-to-go recursively, starting from zero after the last observed reward:

$$G_t = r_t + \gamma G_{t+1}, \qquad G_T = 0.$$

For B complete episodes collected under the same policy parameters, minimize

$$L(\theta) = -\frac{1}{Bc}\sum_{i=1}^{B}\sum_{t=0}^{T_i-1} \gamma^t G_{i,t} \log \pi_\theta(a_{i,t} \mid x_{i,t}).$$

Returns are detached from the gradient. The positive, fixed scale c is 20 for CartPole and 1 for Pong. It changes gradient units, without subtracting anything from rewards or altering the objective's optimum. With Adam's epsilon, changing scale is not perfectly equivalent to changing the learning rate; it is recorded as a hyperparameter. Episodes are averaged, and action terms within an episode are summed. Dividing each episode by its length would give a different weighting of trajectories. No parameters change while collecting a batch. Log probabilities are recomputed from saved observations and actions before the optimizer step, with no dropout or other stochastic layers. Computation is split into chunks of at most 1,024 observations to bound GPU memory, while retaining all decisions.

The outer gamma^t matters for the specified start-of-episode discounted objective. Using only G_t generally gives a different estimator. The exact enumeration experiment below deliberately distinguishes the two. Reward-to-go removes past rewards whose expected score-function gradient is zero; this follows causality and does not introduce an action-independent baseline.

Environments and policies

Setting	CartPole	Pong
Environment	CartPole-v1	ALE/Pong-v5, the modern namespaced Pong-v5
Discount	0.95	0.99
Observation	All four state coordinates, unmodified	Consecutive preprocessed game frames
Network	4 -> 64 tanh -> 2 logits	6,400 -> 64 ReLU -> 2 logits
Action sampling	Categorical, actions 0 and 1	Categorical indices 0 and 1 mapped to actions 2 and 3
Batch	16 complete episodes	16 complete episodes
Adam rate	0.003, after training-only pilots	0.01; weak final performance discussed below

Setting	CartPole	Pong
Training budget	6,000 episodes, fixed before final evaluation	10,000 completed episodes

Both networks start close to a uniform action distribution. Training and required evaluation sample actions from the stochastic policy; greedy evaluation is not silently substituted. Evaluation runs with frozen weights and distinct reset and action-sampling seeds. Training seed is 3817; evaluation reset seeds begin at 1,500,000,000. A shared reset seed does not make two different policies follow the same trajectory. The random reference is evaluated separately.

Pong uses RGB images, the assignment's crop, factor-two downsampling, background removal, and binarization. A copy is made before changing pixels, so the emulator's observation is not mutated. The policy input is current binary image minus the previous binary image, flattened. Values are -1, 0, and 1, conveying motion; the first input of each episode is zero and the previous frame is cleared at reset. This is a compact partial-observation representation, not a claim of a fully observed Markov state. No frames from a previous game leak into a new one.

The v5 defaults are preserved: four emulator frames per decision and sticky action probability 0.25. No additional frame-skip wrapper, FIRE action, no-op warmup, reward reshaping, or life-based episode segmentation is applied. Only environment actions 2 (RIGHT) and 3 (LEFT) are passed to step. A real-game test confirms the game starts and finishes with this action restriction. The emulator's 108,000-frame episode limit is retained. Any truncation is recorded and reported because a truncated game need not have a final winner. These settings follow the official Pong specification.

CartPole results and required plots

The learning-rate pilots each ran 3,000 episodes with the same training seed and other settings. Selection used training data, before held-out evaluation:

Adam learning rate	Last 100 training scores: mean	Last 500: mean
0.01	95.82	164.344
0.003	252.87	246.052
0.001	82.08	78.202

The smaller rate 0.003 gave the best late-training scores in this limited pilot, so it was used for a fresh, fixed 6,000-episode final run. This is a single-seed hyperparameter comparison, not evidence that 0.003 is universally best. The final 100 training scores averaged 166.72, showing that even this rate does not eliminate performance degradation. No best-looking checkpoint was selected after evaluation. For the primary seed, the best 100-episode training window averaged 290.36 and ended at episode 3,097, whereas the final 100 averaged 166.72. More training did not reliably improve this run.

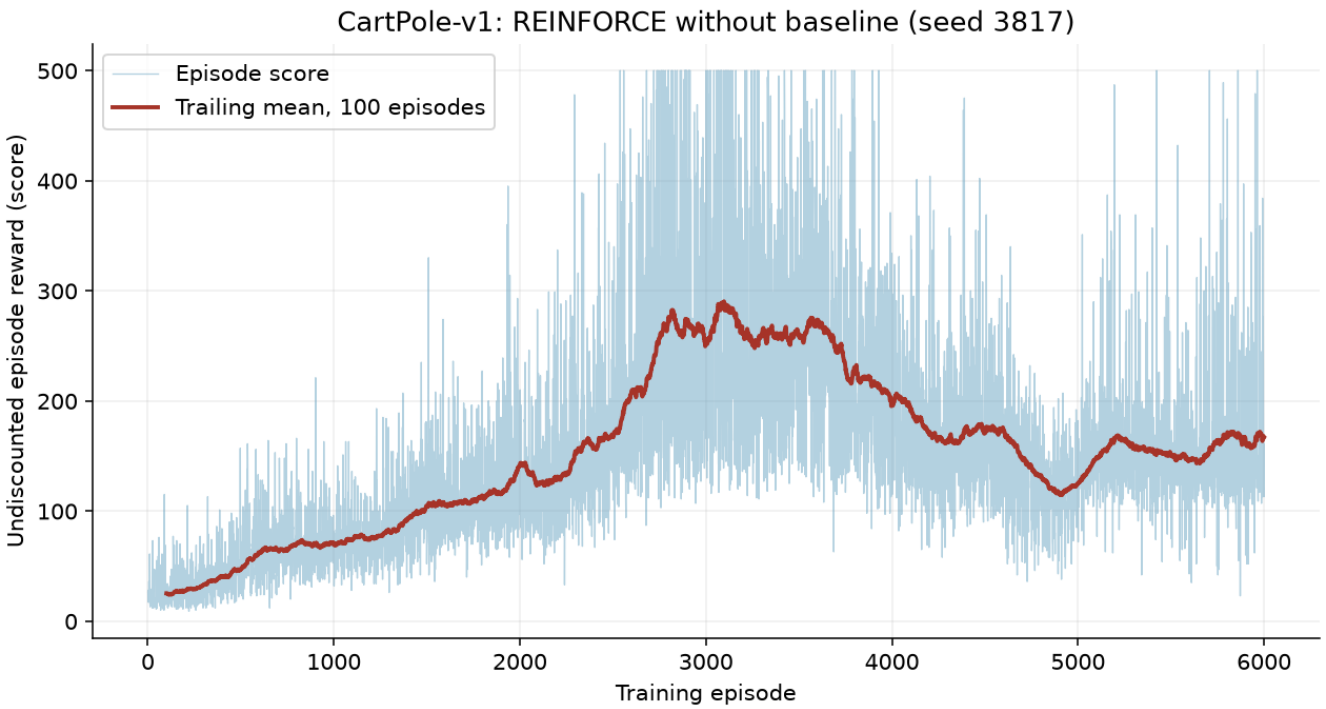


Figure 1. Raw undiscounted episode rewards, with a trailing mean over exactly 100 complete episodes. The mean first appears at episode 100, with no padded or centered window. Full episode scores and discounted returns are in training.csv.

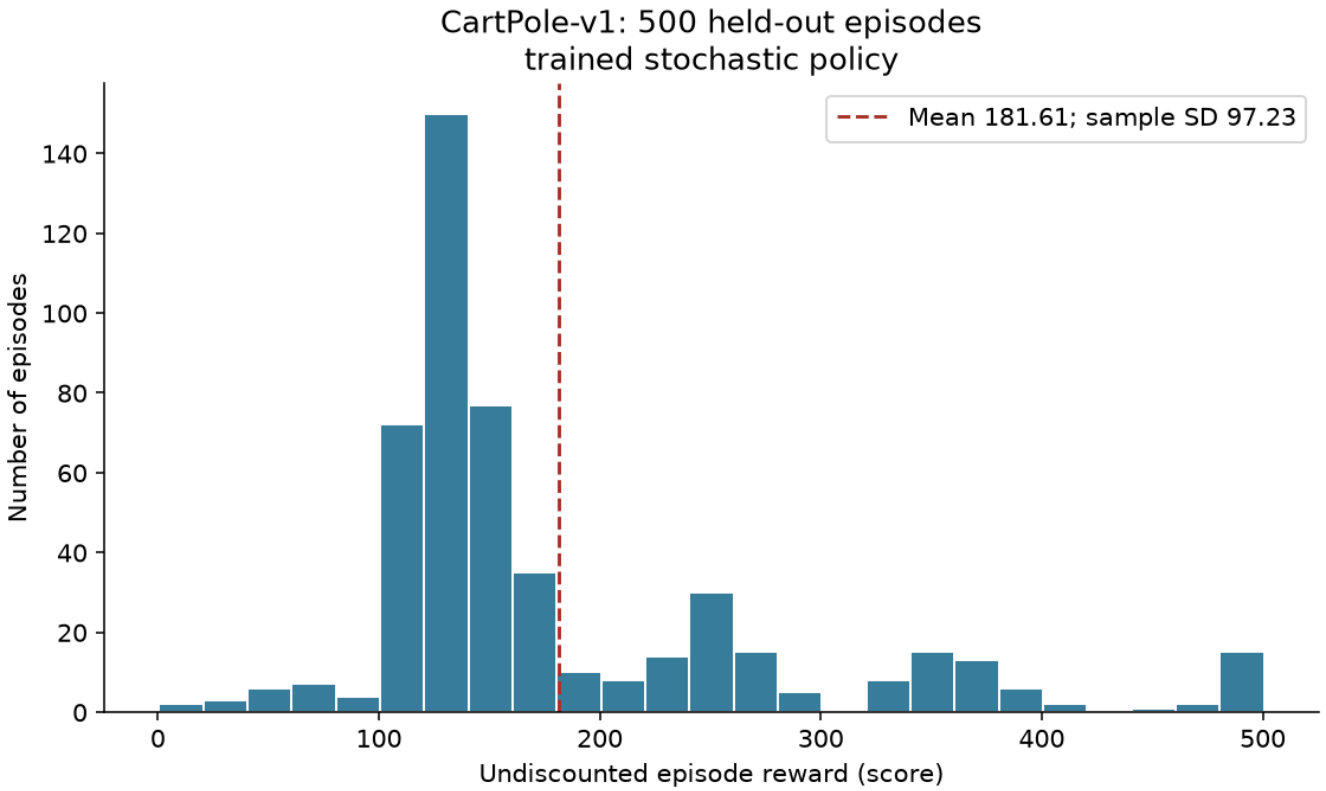


Figure 2. The frozen final stochastic policy evaluated for 500 episodes.

Evaluation policy	Episodes	Mean score	Sample SD	Approximate 95% interval for mean
Final trained policy	500	181.612	97.2304	[173.0894, 190.1346]
Uniform random	500	22.140	12.7489	[21.0225, 23.2575]

The reported SD uses ddof=1; population SD is also saved. Trained scores ranged from 20 to 500. Exactly 15 of 500 episodes reached the 500-step limit (3%); 485 ended in physical failure. The policy improves substantially over random, but **does not solve CartPole reliably**. Its mean discounted return is 19.90266, versus 12.55544 for random. The score histogram exposes variability hidden by a near-saturated discounted objective. Confidence intervals describe evaluation sampling variation for this checkpoint, not training variation across seeds.

Two additional training seeds used the same settings and budget, with 500 frozen evaluation episodes each. The predeclared primary seed remains 3817.

Training seed	Evaluation games	Mean score	Episode-score sample SD	Games reaching 500
3817 (primary)	500	181.612	97.2304	15
3818	500	131.218	23.3961	0
3819	500	174.812	59.8051	0

The mean of these three checkpoint means is 162.5473; their sample SD is 27.3442. This between-run SD differs from the episode-score SD of a fixed checkpoint. All three improve over the recorded uniform reference, but none is a dependable 500-step policy. Initialization/action seeds differ; reset-seed banks overlap, so the three-run summary is descriptive rather than an independent-seed confidence statement. Test results were not used to select a replacement seed.

Pong results and required plots

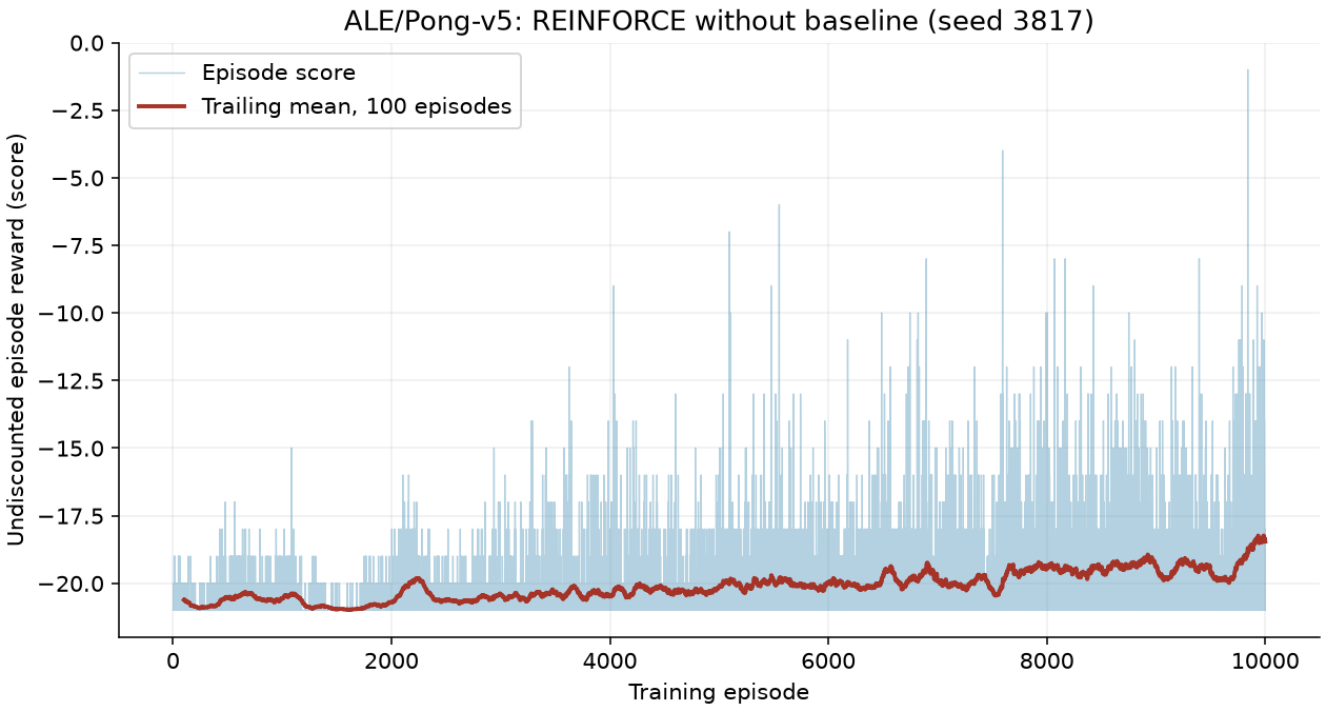


Figure 3. All 10,000 training episodes, with raw scores and the exact trailing 100-episode average. The first 500 scores averaged -20.742, and the last 500 averaged -19.142. The last 100 averaged -18.46. These are descriptive training comparisons involving changing policies, not held-out estimates of one policy.

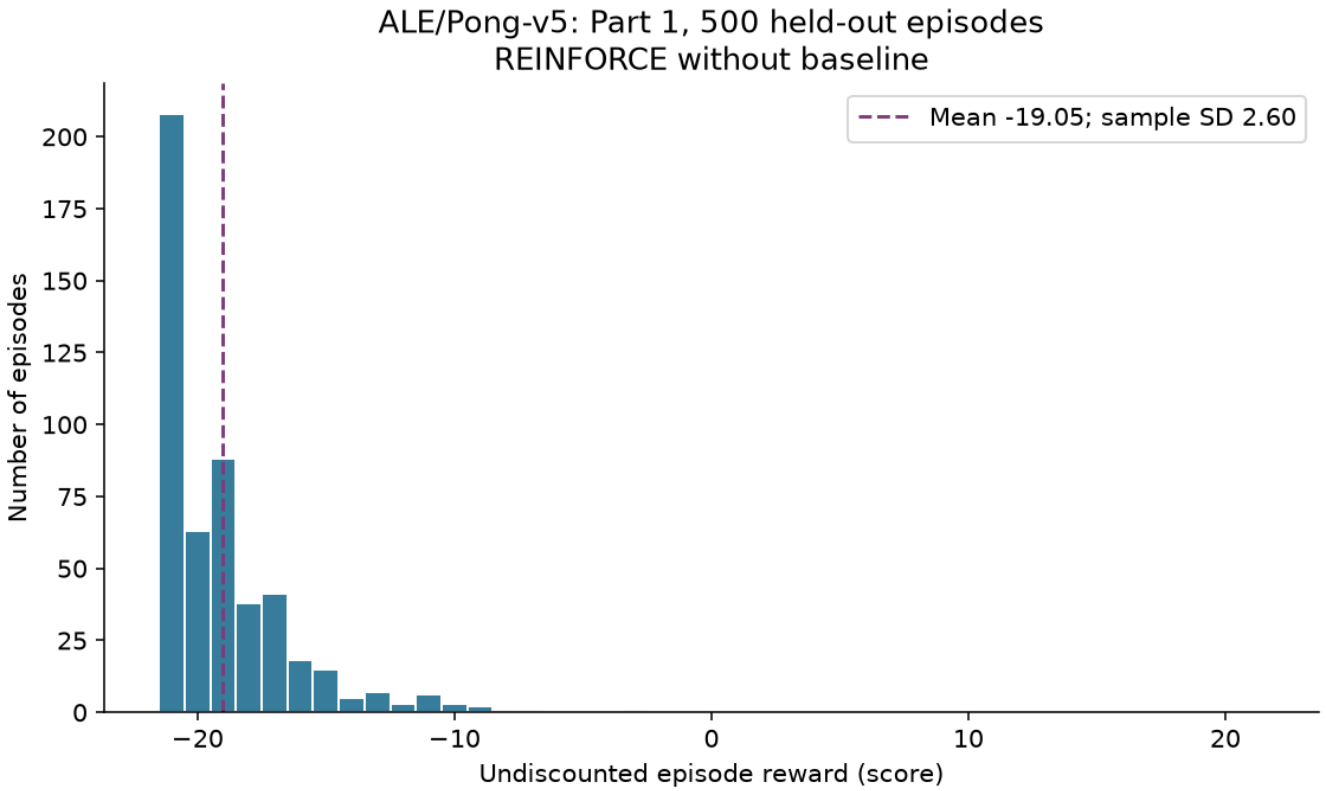


Figure 4. The final frozen stochastic policy over 500 official held-out games. Its mean score is **-19.046**, sample SD **2.5987**, approximate 95% interval for the mean **[-19.2738, -18.8182]**, and range **-21 to -3**. It won **0/500** games; all 500 terminated naturally and none truncated. Exactly 208 games ended at -21. The policy learned to score more often, but it did not learn to win Pong.

The final training-batch action entropy was 0.1065 nats, compared with ln(2) approximately 0.6931 for a uniform binary policy. Poor scores combined with low entropy motivate checking premature action concentration; they do not prove that this caused the failure. A high-value next tuning experiment is matched, training-only learning-rate pilots at 0.01, 0.003, and 0.001, inspecting scores, discounted returns, and entropy. These smaller rates remain untested for Part 1; the separate Part 2 rate screen later in this report did not establish them as fixes. Evaluation data must remain outside that selection.

Fresh fixed-policy diagnostic controls

Existing independent controls used 64 fresh games per policy, reset seeds starting at 1,600,000,000, and independent per-episode action streams. Weights were frozen; these games were not used for training or checkpoint selection.

Policy	Diagnostic games	Mean score	Sample SD	Wins
Final trained policy	64	-18.6563	2.9343	0
Uniform random	64	-20.5781	0.7304	0
Always RIGHT	64	-21.0000	0	0
Always LEFT	64	-21.0000	0	0

These controls support limited learning beyond simple reference policies, while showing no winning policy. They remain separate from the required 500-game evaluation: their scores are not pooled into its histogram or summary. Raw diagnostic scores and summaries are in results/q2/verifier/pong_controls.csv and pong_controls.json. Reproduce them with `python verify_q2_pong_controls.py`. The optional 500-game random Pong reference was omitted to reduce runtime; this smaller, clearly labeled diagnostic supplies a direct comparison.

Edge cases and quantitative validation

Run `python validate_q2.py --pong`. Evidence is saved under results/q2/validation, including CSV comparisons and first-ten-step traces. These experiments test predicted behavior, independently check the objective, and expose task limitations.

1. **Exact multistep gradient.** Enumerate all eight trajectories in a three-step decision tree. Early actions affect later states and rewards, including negative rewards. Differentiate the exact expected discounted return directly, and compare it with the probability-weighted REINFORCE estimator. Across all seven state logits, maximum disagreement is 2.78e-17. Omitting outer discounting gives error 0.21854. This checks temporal credit and the stated objective, rather than relying on visual improvement in a noisy curve.

2. **Learning where the optimum is known.** In a one-step stochastic task, action reward probabilities are 0.2 and 0.8. Starting uniformly, the same categorical loss with Adam 0.01 learns best-action probabilities 0.97787, 0.97897, and 0.97974 in three independent runs. Exact expected rewards are 0.78672–0.78784, versus 0.5 for random. This tests exploration, gradient sign, and learning under noisy rewards without relying on the complex game environment.

3. **Discount limits, delayed and signed rewards.** For rewards [1,-2,3] and gamma=0.5, reward-to-go is [0.75,-0.5,3] and gradient weights are [0.75,-0.25,0.75]. At gamma=0 only the first action receives credit; at gamma=1 the undiscounted future sums are recovered. A zero-reward episode gives exactly zero gradient without division by zero. Punishing action 1 decreases its probability from 0.5 to 0.49500 after one Adam step. Very large logits remain finite through categorical log probabilities, without manually taking log(0).

4. **Sparse Pong rewards and point boundaries.** For [1,0,-1], gamma=0.99 gives returns [0.0199,-0.99,-1], demonstrating that the next point's outcome is retained. A reward delayed 1,000 steps gives every earlier eligible action weight 0.99^1000 = 4.3171e-5. Discount is never reset when a point is scored. Float64 return calculations preserve these small values longer; they cannot prevent genuine vanishing credit over long games. At step 5,000 the absolute discount is approximately 1.50e-22. This is a limitation of the specified start-discounted objective, not a reason to silently train on a different one.

5. **CartPole physics and reset reproducibility.** Independent one-step cart-pole equations agree with the environment within 1.71e-9. Repeating an episode with identical reset and action RNG seeds reproduces observations and actions exactly. The first-ten-step JSON records state, action probability, action, reward, next state, reward-to-go, and gradient weight. A uniform-policy 100-episode check averages 21.54 points (SD 10.8901), consistent with the larger independent reference evaluation above.

6. **Termination versus truncation.** The collector stops after either flag, retains the last reward, and resets before another action. A diagnostic one-step CartPole TimeLimit ends by truncation, with length and return exactly one. This deliberately modified environment is used only in validation. The assignment run keeps CartPole-v1's 500-step cap. With a complete episodic Monte Carlo objective over the observed game, no critic bootstrap is added at the cap. Pong truncations are counted separately from completed wins/losses.

7. **Pong pixels, motion, actions, and a full real game.** A synthetic frame has exactly one surviving foreground pixel after preprocessing; the original input remains unchanged. Moving it produces one -1 and one +1 motion pixel, and a fresh episode produces zero motion. A full real v5 rollout using only actions 2/3 terminates after 888 decisions with agent/opponent points 0/21, hence score -21. Points occur at decisions 63 through 887; their signed rewards and gradient weights are saved. The action labels, frame skip, and sticky-action probability are checked against the actual emulator.

8. **Interruption and replay.** Compare a 16-episode CPU training run with an identical run stopped after episode eight and resumed from its checkpoint. All episode scores, lengths, and terminal flags agree and the maximum parameter difference is exactly zero. Optimizer state, action RNG state, episode count, and explicit reset seeds preserve the experiment. This is same-device evidence; cross-device bitwise equivalence is not assumed. The same interruption experiment also passed on Deepdish5's RTX PRO 5000 GPU, with exactly zero parameter difference. Its independent validation evidence is saved under results/q2/deepdish/validation. The GPU identity and one-device visibility were verified; server-specific selection details are in DEEPDISH.md.

9. **Learning stability and frozen evaluation.** The three learning-rate pilots expose late-training instability rather than hiding it behind a peak score. The final evaluation keeps weights fixed, samples the trained stochastic policy, uses 500 held-out reset seeds, and compares against a 500-episode uniform reference. Evaluation data were not used to choose the learning rate. The three completed CartPole seeds and fresh frozen Pong controls above add task-specific evidence. Broader independent seed replication is still needed to estimate training reliability.

10. **Independent completed-run audit.** Existing evidence verifies all four evaluation checkpoint hashes, the 10,000 Pong episodes and 625 updates, and disjoint training/evaluation reset ranges. The first 5,376 episode records and 336 updates exactly match the archived serial run; its checkpoint hash matches continuation metadata. Part 1 has no baseline state. An independent variable-length finite-difference gradient check agrees within 1.57e-11, and the production chunked Adam update agrees with an unchunked update within 9.32e-10 in parameters. Serial/batched partial Pong trajectories and score-only reset handling agree, and the first two official GPU evaluation games replay exactly on CPU, including lengths and ending flags. This evidence is preserved under results/q2/verifier; it was incorporated without repeating training. Reproduce the mathematical/update checks with `python verify_q2_independent.py`, and the completed Pong collector replay with `python verify_q2_completed_collector.py`.

For CartPole's constant reward, a 100-step episode already has discounted return 19.88159, while a 500-step episode has return almost exactly 20. Thus gamma=0.95 strongly compresses differences between medium and long survival. This helps explain why a mathematically correct estimator can improve discounted return without reliable 500-step scores. A rising curve alone would not have revealed that distinction. Similarly, Pong's sparse rewards, partial observation, sticky actions, and negative early scores make baselineless Monte Carlo gradients noisy.

Execution and reproducibility

Use Python 3.11+ with requirements-q2.txt. The tested local environment used Python 3.13.13, PyTorch 2.15.0, Gymnasium 1.4.0, ALE 0.12.1, NumPy 2.5.4, and Matplotlib 3.11.2. Every run saves its actual versions and configuration. Deepdish should use a compatible CUDA-enabled PyTorch installation and expose exactly one GPU. See README.md and DEEPDISH.md for complete commands and output locations.

Reproduce the primary CartPole training and its required evaluation in a fresh directory (CPU is the default):

```
python q2_policy_gradient.py train --game cartpole --episodes 6000 --lr 0.003 \
  --output-dir results/q2/cartpole/reproduction
python q2_policy_gradient.py evaluate \
  --checkpoint results/q2/cartpole/reproduction/checkpoint.pt \
  --episodes 500 --output-dir results/q2/cartpole/reproduction/evaluation_trained
```

The optional random reference uses the same evaluate command with `--random-policy` and a different output directory. To repeat the additional training seeds, add `--seed 3818` or `--seed 3819` to train and choose a fresh directory for each run. The evaluator reads the training configuration from the checkpoint; it performs no training.

This report covers Policy Gradient Part 1. The shuttle and baseline experiments are documented separately. All experiment source, figures, CSVs, metadata, and independent verification code are published in the repository identified above.

The serial/batched transition changes the action RNG protocol. To reproduce the reported two-stage Pong experiment on one assigned GPU from a fresh directory:

```
python q2_policy_gradient.py train --game pong --episodes 5376 --lr 0.01 \
  --reward-scale 1 --device cuda --output-dir results/q2/pong/reproduction
python fast_pg.py train --part 1 --game pong --episodes 10000 --lr 0.01 \
  --reward-scale 1 --batch-episodes 16 --workers 8 --save-every 4 --device cuda \
  --resume results/q2/pong/reproduction/checkpoint.pt \
  --output-dir results/q2/pong/reproduction
python fast_pg.py evaluate --checkpoint results/q2/pong/reproduction/checkpoint.pt \
  --episodes 500 --workers 8 --device cuda \
  --output-dir results/q2/pong/reproduction/evaluation_trained
```

Set CUDA_VISIBLE_DEVICES to the one assigned card and CUBLAS_WORKSPACE_CONFIG=:4096:8 before CUDA execution. Use the recorded package versions for the closest replay; cross-device bitwise agreement is not assumed. To reproduce the continuation without repeating the first stage, copy the archived checkpoint at legacy_snapshots/through_episode_5376/ to a new run directory, then use the last two commands with that checkpoint path. A fresh 10,000-episode serial run would use a different action stream after episode 5,376 and is not an exact reproduction of this experiment.

Recorded runtime continuation

After episode 5,376, the Part 1 checkpoint, source, and logs were archived and training resumed with a validated batched collector. Eight CPU workers feed one GPU, complete all 16 trajectories under fixed policy parameters, and then perform the same actor update. In an isolated remote benchmark all 32 game scores and lengths matched across serial CPU/GPU and batched collectors; actor updates also matched in worker-count validation. The new action RNG protocol and source hashes are recorded. The continuation is statistically on-policy, but is not a claim of bitwise replay of the legacy serial action stream. Training completed at 10,000 episodes and frozen evaluation at 500; the optional random reference was omitted. Checkpoints now save every 64 episodes and at completion. See DEEPDISH.md and results/q3/collector_benchmark_deepdish.json for timing and validation evidence.

With three additional months

Month 1: estimator and optimization reliability. Repeat training with at least five independent seeds for every main setting. Predeclare episode budgets, pilot/evaluation seed splits, and learning-rate comparisons. Compare batch sizes and fixed reward scales while checking the exact objective and gradient direction. Report distributions of final evaluation performance and wall-clock/sample cost, not just the best run. Extend exact-gradient enumeration to variable-length episodes and independently compare finite differences. Check CPU versus single-GPU agreement and resumed versus uninterrupted training on the actual GPU backend.

Month 2: observation and environment sensitivity. Within Pong-v5 compare frame differences with two-frame stacks and compact convolutional policies under matched budgets. Keep v5 sticky actions and frame skipping unchanged in required runs; perform any deterministic or altered-discount variants as explicitly labeled diagnostics. Study how frame timing, delayed rewards, and truncation frequency affect learning. For CartPole evaluate recovery from controlled near-boundary states separately from the official initial-state distribution, including pole angle and velocity shifts. Test whether high scores persist across resets and whether the input representation loses useful information.

Month 3: robustness, variance, and generalization. Compare Part 1 against Part 2 baselines using matched seeds, environment settings, and budgets. Measure gradient variance as well as score variance, and report confidence intervals across training seeds. Evaluate fixed checkpoints on new, untouched seeds, including greedy rollouts as a separately labeled secondary measure. Track Pong point-win probability, action entropy, episode length, and timeouts to distinguish learning to rally from learning to win. Repeat longer runs to quantify catastrophic degradation and evaluate checkpoint-selection rules using a separate validation set without reusing the final 500-episode test set.

Policy Gradient: Part 2

This section repeats the two Part 1 experiments with a baseline (20 points: 10 for plots, 10 for edge cases and tests). CartPole training and the required 500-game evaluations are complete, as are the full Pong reference experiment and its 500-game evaluation. The required baseline explanation, four game-specific plots, quantitative validation, edge cases, and three-month experiment plan are included below. Pong failed to learn a winning policy; this limitation remains explicit in the final results.

Completed configuration	Training episodes	Evaluation games	Mean score	Sample SD
CartPole, EMA baseline, primary seed	6,000	500	438.342	95.7679
Pong, EMA baseline, matched reference	10,000	500	-19.460	0.9952
Pong, EMA baseline, CNN follow-up	4,480	500	-18.492	1.8722

The first two rows are the required repeated experiments; the CNN is an additional tuning experiment. Both Pong configurations lost all evaluation games.

Baseline and implementation

I use an adaptive scalar baseline: a bias-corrected exponential moving average of **previous batches' episode-start discounted returns**. It is constant across states, actions, and decision times within the current batch, and changes only between batches. This is a simple constant baseline rather than a learned critic. It adds a scalar update, with negligible cost compared with game simulation.

Let m_k be the mean G_0 among the complete episodes in batch k. Initialize $n_0 = z_0 = 0$ and $b_0 = 0$. With decay $d = 0.9$, after the actor update for batch k compute

$$n_{k+1} = dn_k + (1-d)m_k, \quad z_{k+1} = dz_k + (1-d), \quad b_{k+1} = n_{k+1}/z_{k+1}.$$

The denominator corrects the startup shrinkage toward zero. The next batch uses b_{k+1}. The current batch uses b_k, fixed before its trajectories are sampled. Using the old value avoids the finite-sample bias that can arise when a trajectory helps determine the baseline subtracted from its own return.

The actor still maximizes the assigned episodic objective $J(\theta) = \mathbb{E}[\sum_t \gamma^t r_t]$. Its loss is

$$L(\theta) = -\frac{1}{Bc}\sum_{i=1}^{B}\sum_{t=0}^{T_i-1}\gamma^t(G_{i,t} - b_k)\log\pi_\theta(a_{i,t}|x_{i,t}).$$

The baseline and returns are detached. The fixed scales $c = 20$ for CartPole and $c = 1$ for Pong match Part 1. No advantage standardization, critic bootstrap, entropy bonus, pointwise discount reset, or reward reshaping is introduced. Conditioning on the previous training history and current state, b_k is fixed, and $\mathbb{E}_a[\nabla_\theta \log\pi_\theta(a \mid s)] = \sum_a \nabla_\theta\pi_\theta(a \mid s) = 0$. Thus subtracting b_k leaves the expected gradient unchanged, including when episodes end at different times. It can change variance and individual optimizer steps, as intended.

A return-average baseline is not generally variance optimal; this is a known limitation of baseline control variates, discussed by Greensmith, Bartlett, and Baxter (2004). I measure the effect directly, rather than infer it from a smoother reward plot. The exact counterexamples below also identify when this simple choice can fail.

All actor settings match Part 1: CartPole-v1 gamma=0.95 and Adam=0.003; ALE/Pong-v5 gamma=0.99 and Adam=0.01; 64 hidden units, two output logits, 16 complete episodes per update, stochastic actions, and the same fixed reward scales. Pong retains the 80x80 frame differences, actions 2/3 only, v5 frame skip 4, sticky-action probability 0.25, and the official episode limit. CartPole has 6,000 completed training episodes; Pong has 10,000. Every run starts from a fresh policy, rather than warm-starting Part 2 from the Part 1 trained network.

CartPole results

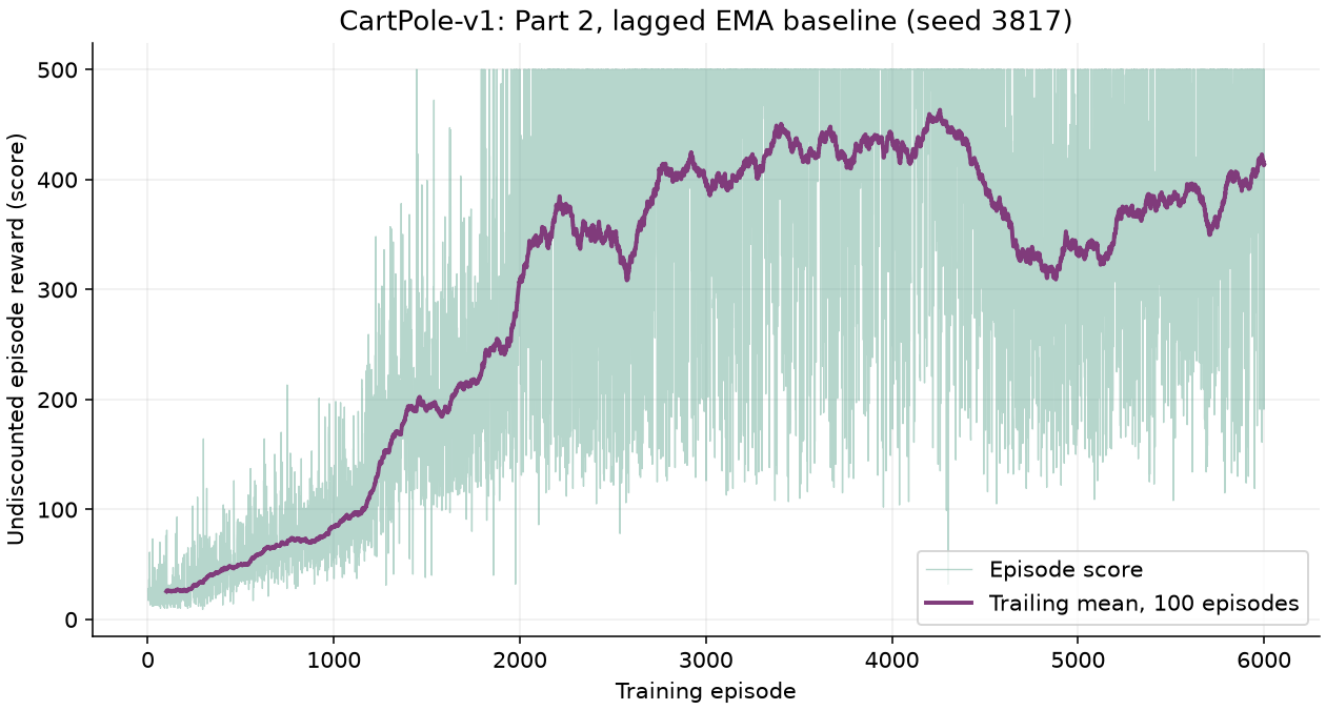


Figure 1. Primary training seed 3817. The trailing mean uses exactly 100 episodes and starts at episode 100. No shortened or padded window is shown.

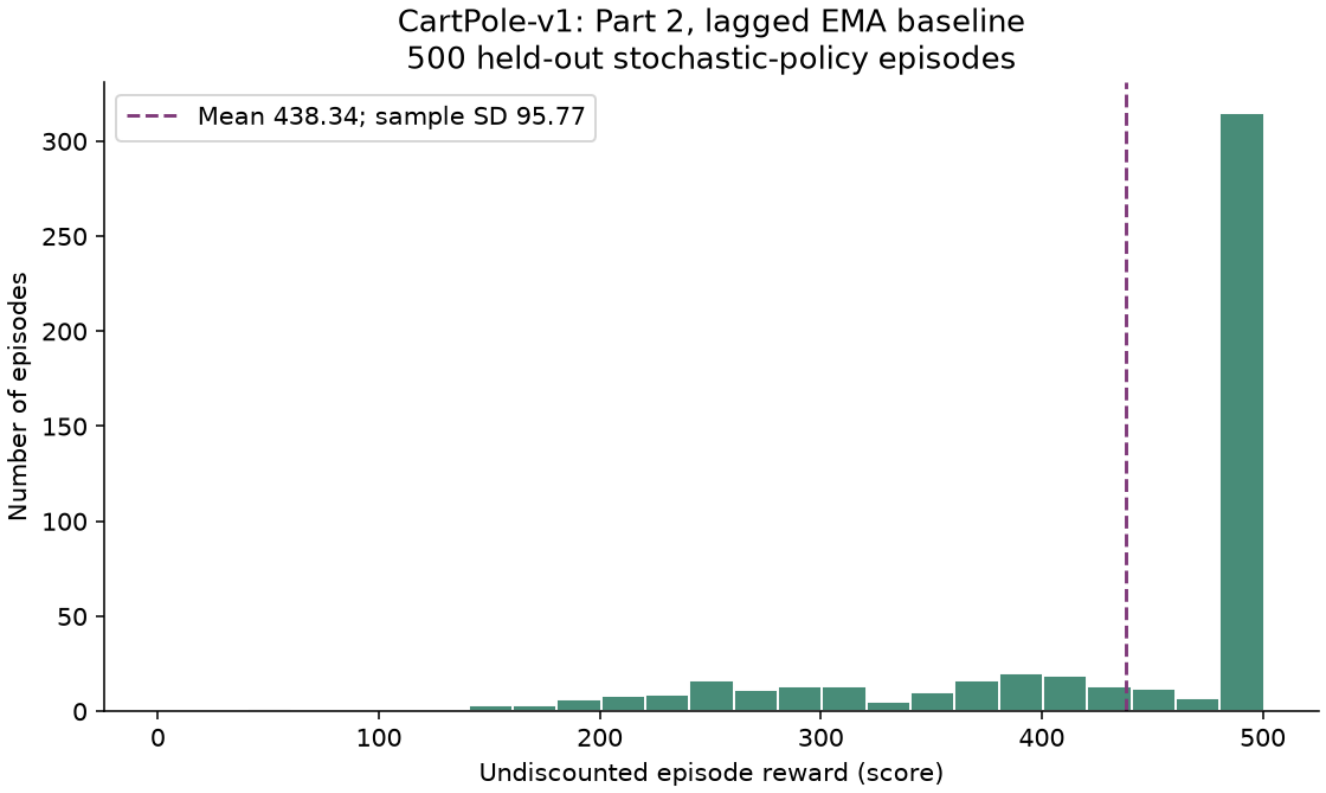


Figure 2. Frozen final stochastic policy, with 500 held-out episodes. The mean is **438.342**, sample SD **95.7679**, and approximate 95% interval for this checkpoint's mean [**429.9476, 446.7364**]. Scores range from 46 to 500. 303/500 episodes reach 500 (60.6%). The policy improves substantially over Part 1, but is not reliably perfect. The test set was not used to select a checkpoint, baseline decay, or learning rate.

Three matched training initialization/action seeds give the following results. Each row uses a fixed 6,000-episode training budget and 500 evaluation games.

Seed	Part 1 mean	Part 1 sample SD	Part 2 mean	Part 2 sample SD	Part 2 fraction reaching 500
3817	181.612	97.2304	438.342	95.7679	0.606
3818	131.218	23.3961	434.470	102.0657	0.602
3819	174.812	59.8051	388.232	104.6479	0.304

The mean of the three final evaluation means increases from 162.5473 to 420.3480. The sample SD across those three means is 27.3442 for Part 1 and 27.8806 for Part 2. Thus this small replication supports improved scores, not reduced between-training variability. It also demonstrates that smaller **gradient** variance need not mean smaller **episode-score** SD: seed 3818's score SD rises. The training seeds use different initialization and action streams, but their reset-seed banks overlap. These are descriptive paired comparisons; no claim of an independent-seed confidence interval or universal baseline superiority is made. Termination and truncation flags may both be true on the same final transition; the saved counts are not assumed mutually exclusive.

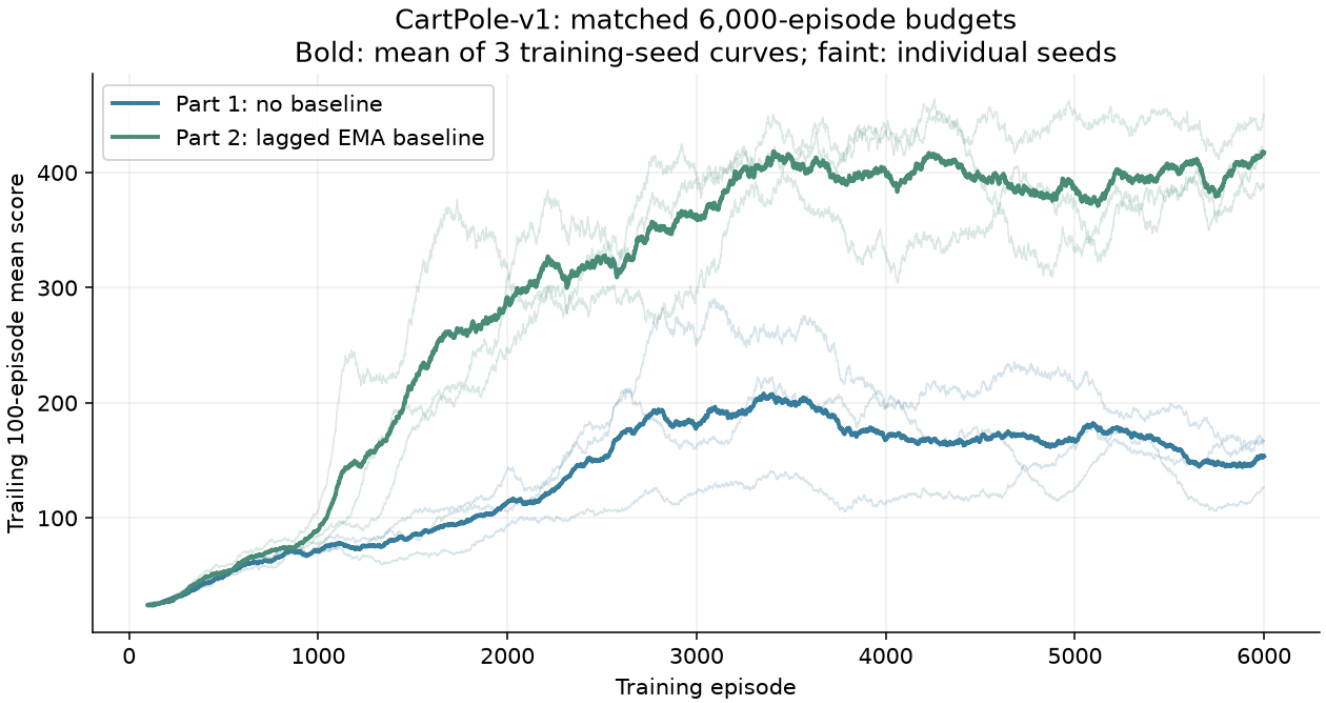


Figure 3. Equal training budgets, with all three seeds retained. Bold curves average the three trailing means; faint curves show individual training runs.

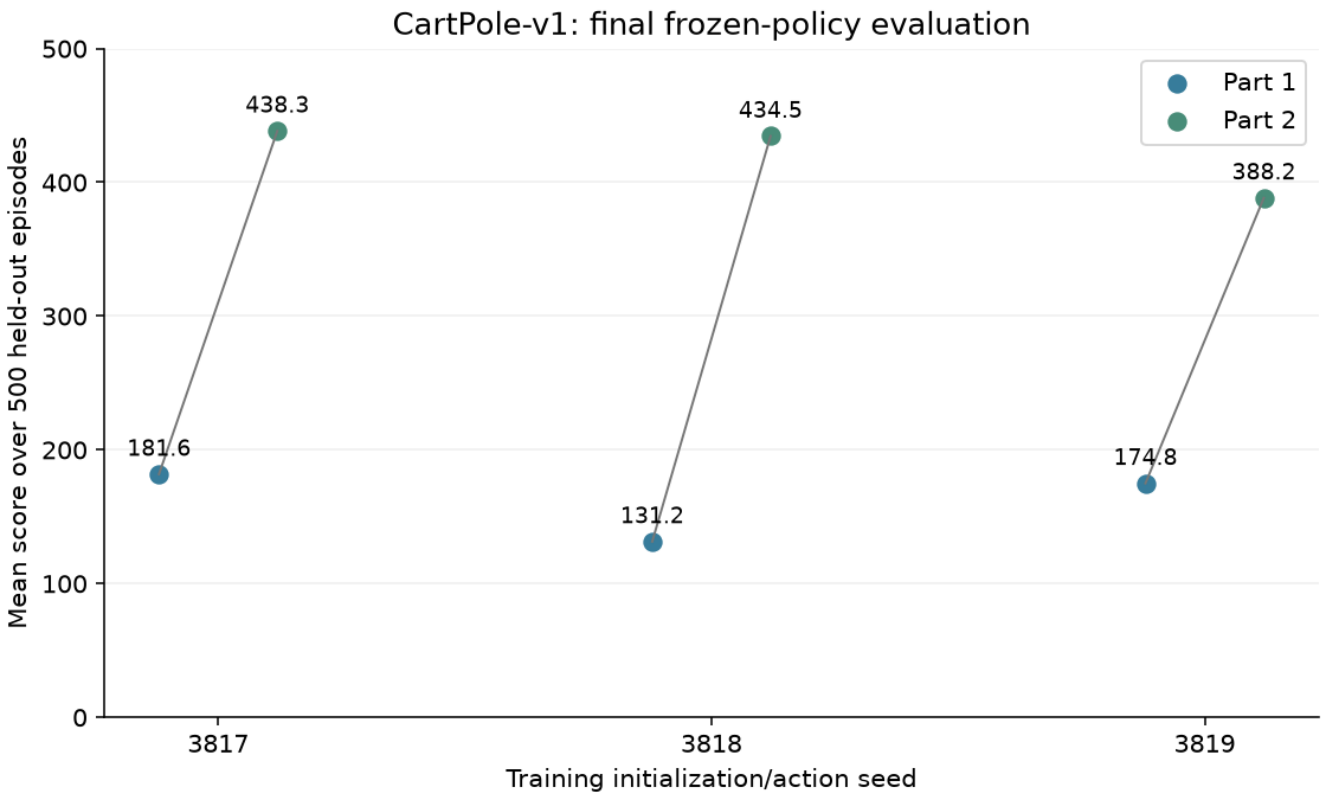


Figure 4. Each point is one frozen checkpoint's mean across 500 evaluation episodes. This preserves training-seed differences rather than pooling 1,500 games and treating them as one independently trained policy.

Gradient-variance experiments

At each frozen policy, estimate the baseline on independent calibration games, freeze it, then compute both gradient estimates on the **same** probe trajectories. Measure the trace of the sample gradient covariance with ddof=1. Policies and observations are identical within each pair. Thus the comparison isolates the baseline, rather than confounding it with an already improved policy.

Frozen policy	Calibration / probe episodes	Baseline	Raw gradient variance	Baseline gradient variance	Ratio and paired-bootstrap 95% interval
Initial CartPole	64 / 128	13.1001	0.97880	0.94067	0.9610 [0.7148, 1.2498]
Final Part 1 CartPole	64 / 128	19.9725	18.08717	0.30767	0.01701 [0.0000615, 0.05208]
Initial Pong	32 / 32	-1.44340	124.50566	15.40717	0.12375 [0.09414, 0.16511]

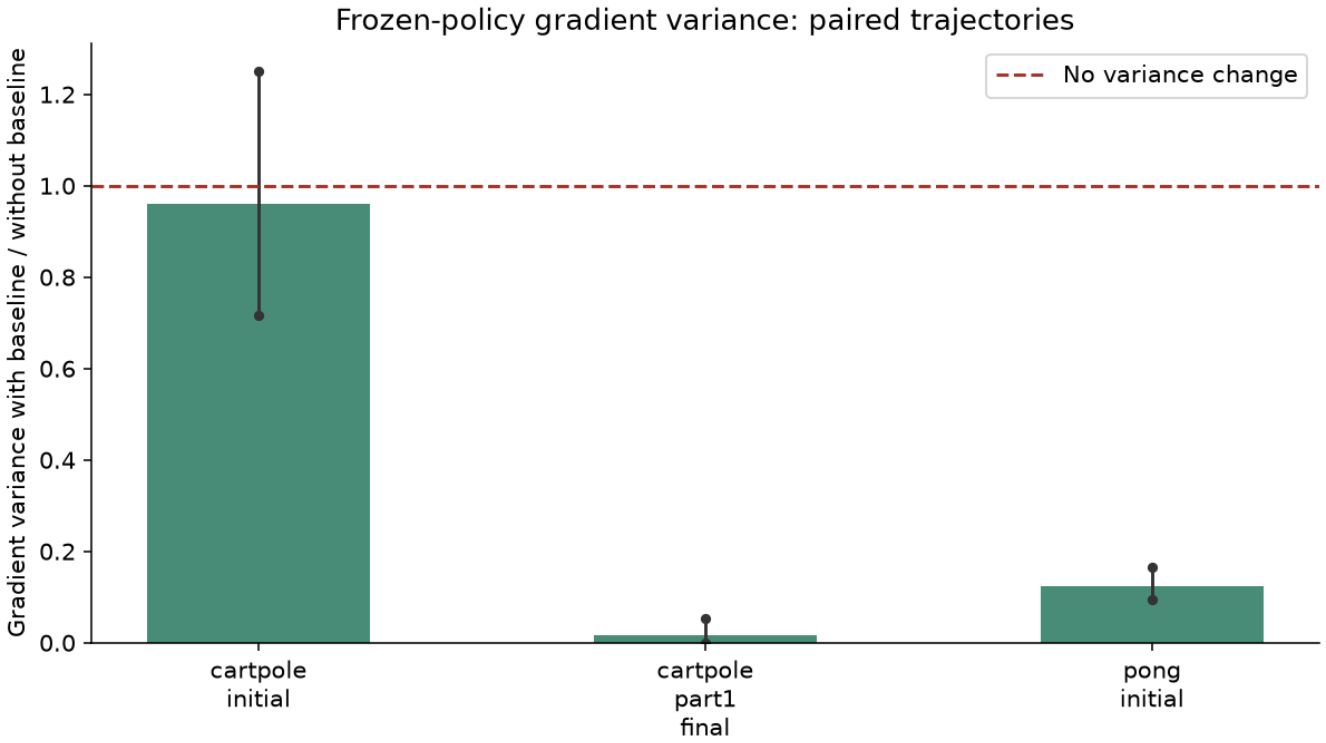


Figure 5. Ratios below one indicate variance reduction. There are 2,000 paired bootstrap resamples; both estimators use the same trajectory indices in every resample. The initial CartPole result is inconclusive. The final CartPole policy shows about 98.3% reduction, and initial Pong about 87.6%. These intervals are conditional on the chosen frozen policy and fitted baseline, and do not include training-seed or baseline-fitting uncertainty. Pong's small sample warrants replication. Absolute variances across games should not be compared because parameter dimensions and reward scales differ.

Edge cases and further validation

Run `python validate_q3.py --variance --pong`. The validation JSON and CSV files under results/q3/validation preserve all predictions and measurements.

1. **Exact baseline invariance.** Enumerate eight three-step trajectories and differentiate the exact expected discounted reward directly. Constant baselines -10, 0, and 5 leave the expected REINFORCE gradient unchanged to below 1e-12. A separate five-trajectory example ends immediately after one root action, exercising variable episode length and termination. It also agrees below 1e-12. An intentionally action-dependent baseline changes the gradient by 0.24026 in the fixed-length example, demonstrating that the experiment detects invalid baseline dependencies.

2. **Same-batch centering bias.** In an exactly enumerated Bernoulli bandit with p=0.3 and reward equal to action, the true gradient is 0.21. Subtracting the current batch's mean reward gives expected gradient 0 for batch size one and 0.1575 for size four, the factor (1-1/B). Our previous-batch update avoids this dependency. This is why the baseline update order is part of the algorithm.

3. **Known variance reductions and failures.** For deterministic rewards and p=0.3, the no-baseline gradient variance is 0.1029; the return-mean baseline reduces it to 0.0336, and the variance-optimal constant reduces it to numerical zero, while preserving gradient 0.21. In a noisy bandit with p=0.98, the mean baseline increases variance from 0.00401690 to 0.00963881. A baseline is not an automatic guarantee of improvement. This counterexample motivates measuring the actual frozen-game gradient variance rather than asserting all baselines reduce it.

4. **Startup, zero rewards, negative rewards, and discount limits.** The first batch uses zero, so the first actor update must match Part 1. A first baseline target of 20 yields 20 after startup correction. Repeated

zero targets keep b=0 without dividing by a sample SD; negative targets yield a negative baseline. With gamma=0, only the first decision receives gradient credit. A positive reward below the baseline correctly decreases the sampled action's probability. A stale nonzero baseline can add noise even when all current rewards are zero: the exact zero-reward example has expected gradient zero but variance 0.84. Returns are detached, and the baseline is never differentiated through the actor.

5. **Learning with a known optimum.** In the noisy 0.2-versus-0.8 reward bandit, three training seeds learn best-action probabilities 0.98007, 0.97784, and 0.97948. Corresponding exact expected rewards are approximately 0.787–0.788, versus 0.5 for uniform actions. This checks that the lagged-baseline loss and update order still learn under stochastic rewards.

6. **Replay and a controlled first update.** A 16-episode run stopped after eight episodes and resumed exactly reproduces the uninterrupted policy, all episode records, baseline state, and optimizer diagnostics. Part 1 and Part 2's first zero-baseline batch also produce exactly identical policy parameters. GPU replay is validated once before Part 2's remote run, with a cache keyed by source, software environment, and GPU identity.

7. **Environment and runtime edge cases.** Part 1's action mapping, official versions, frame-history resets, final rewards, and termination/truncation handling are reused. The new batched collector preserves every complete episode, including slow games; it does not restart early-finishing games to fill a batch. One-versus-four-worker Pong trajectories and actor updates match exactly. Repeated resets and a partial final batch match the reference. A score-only 17-game evaluation records exactly 17 games. Fast training replay reproduces parameters exactly even when the number of CPU workers changes. Continuing an old checkpoint preserves its prior episode history.

8. **Audit of the completed runs.** `python verify_q3_completed.py` independently reconstructs every EMA from saved returns, without importing the training implementation or running any games. Across three CartPole runs, the Pong reference, and the selected CNN, all 2,030 actor updates agree within 1.43e-14. It checks 80 saved decision records, recomputes all five 500-game score summaries, checks nonoverlapping training/evaluation reset banks, and matches each evaluation's checkpoint SHA-256. Evidence is in results/q3/verifier/completed_artifact_audit.json. This verifies consistency of saved evidence; algorithmic correctness is supported separately by the exact-gradient, variance, learning, and replay experiments above.

Ten-step credit-assignment example

For the first CartPole episode after the initial batch, the baseline is 13.6720064, computed from the preceding 16 episodes. The initial state (cart position, velocity, pole angle, angular velocity) is approximately (-0.001779, -0.042984, -0.007022, -0.012988). Every displayed reward is +1. The policy stays fixed throughout the episode; its probabilities change because the state changes. The last column is gamma^t(G_t-b)/20, before averaging over the 16 episodes in the actor update.

Step	Pole angle	Action	P(action)	G_t	G_t-b	Weight
0	-0.00702	0	0.5015	15.4813	1.8093	0.09046
1	-0.00728	1	0.5019	15.2435	1.5715	0.07464
2	-0.00173	0	0.5016	14.9931	1.3211	0.05962
3	-0.00208	1	0.5018	14.7296	1.0576	0.04534
4	0.00341	0	0.5016	14.4522	0.7802	0.03177
5	0.00304	1	0.5018	14.1602	0.4882	0.01889
6	0.00854	0	0.5016	13.8529	0.1809	0.00665
7	0.00820	0	0.4981	13.5293	-0.1427	-0.00498
8	0.01378	0	0.4947	13.1888	-0.4832	-0.01603
9	0.02526	1	0.5087	12.8303	-0.8417	-0.02652

At step 7 the return falls below the frozen baseline despite a positive immediate reward, so that decision's contribution discourages its sampled action. The full update sums contributions from all decisions and episodes; the sign of one term does not determine the eventual parameter change. Full states, next states, rewards, and ending flags are in results/q3/cartpole/final_seed3817/first_10_steps_after_baseline.json.

Pong reference result and runtime

Pong Part 2 trained a fresh policy for 10,000 episodes on the same single GPU, using the same actor settings and the lagged scalar baseline. Its final 500-game stochastic evaluation averaged **-19.460**, with sample SD **0.9952**, approximate 95% interval [**-19.5472, -19.3728**], and range **-21 to -15**. All 500 games terminated naturally, none truncated, and **none were wins**.

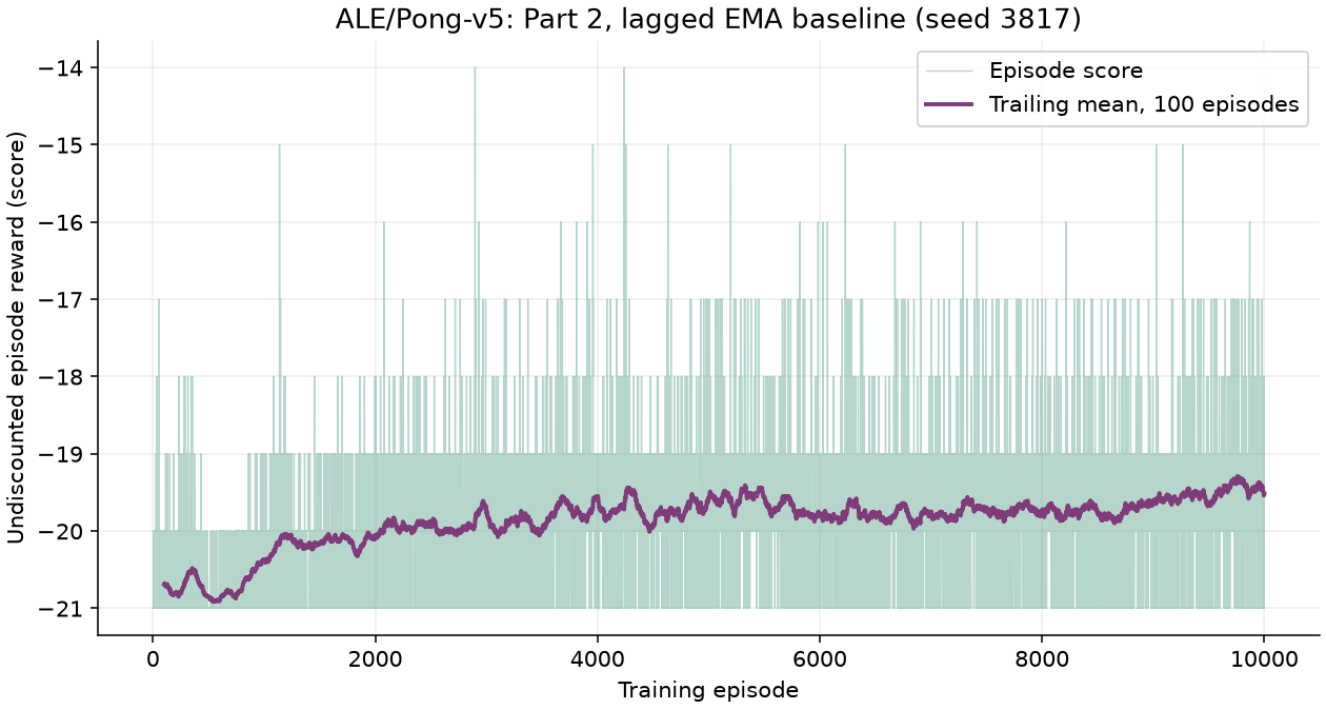


Figure 6. The complete Part 2 reference run. Lower gradient variance did not produce effective game-level learning with these settings.

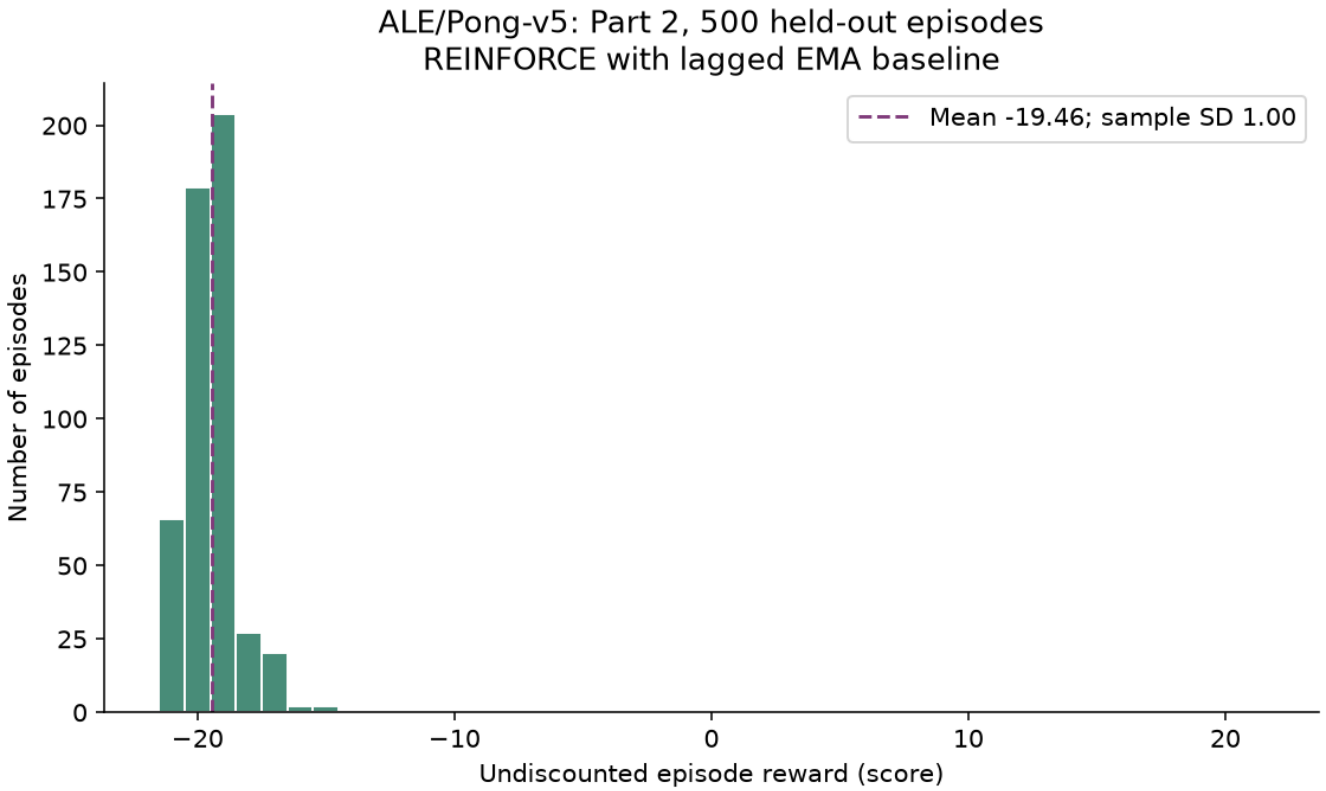


Figure 7. The required frozen-model evaluation. Its mean is worse than Part 1's -19.046. Its discounted return mean, however, is 0.12595 versus Part 1's -0.26569. Improvement in the assigned discounted objective did not translate to winning games. This is a failed training configuration, not a successful Pong model. This failure motivated the controlled training-only experiments reported below; their checkpoints were selected without using the final test games.

The fast collector uses eight CPU workers and batches inference on one GPU. A remote fixed-policy benchmark measured 3,007 decisions/second with sequential CUDA inference versus 13,305 with eight workers, a 4.4x rollout-throughput increase. All benchmark modes produced identical scores and lengths for 32 games. This is a collector benchmark, not a guarantee of a 4.4x end-to-end training speedup. The optional 500-game random evaluation is omitted, while the required trained evaluation remains 500. Checkpoints are saved every four actor batches (64 episodes) and at completion. This reduces I/O but increases interruption replay to at most 64 episodes. Changed code/environment/device triggers revalidation.

Part 1's legacy checkpoint and source were archived before its runtime switch. The new collector uses independent per-episode action RNG streams and records the transition and source hashes. The mathematical policy and objective are preserved; bitwise continuation of the old serial sampling stream is not claimed. This engineering change and single-seed Pong comparison limit causal conclusions.

Execution and reproducibility

Use the same requirements-q2.txt environment as Part 1. Typical local commands:

```
python validate_q3.py --variance --pong
python validate_fast_pg.py
python verify_q3_completed.py
python compare_policy_gradients.py
python q3_policy_gradient.py train --game cartpole --episodes 6000 --lr 0.003 \
  --output-dir results/q3/cartpole/reproduction
python q3_policy_gradient.py evaluate \
  --checkpoint results/q3/cartpole/reproduction/checkpoint.pt \
  --episodes 500 --output-dir results/q3/cartpole/reproduction/evaluation_trained
```

Reproduce the matched Pong reference from a fresh policy and output directory:

```
python fast_pg.py train --part 2 --game pong --episodes 10000 --lr 0.01 \
  --reward-scale 1 --batch-episodes 16 --workers 8 --save-every 4 --device cuda \
  --output-dir results/q3/pong/reproduction
python fast_pg.py evaluate --checkpoint results/q3/pong/reproduction/checkpoint.pt \
  --episodes 500 --workers 8 --device cuda \
  --output-dir results/q3/pong/reproduction/evaluation_trained
```

Before CUDA execution, set the following variables, replacing the quoted GPU identifier with your one assigned card:

```
export CUDA_VISIBLE_DEVICES="your-assigned-GPU-UUID"
export CUBLAS_WORKSPACE_CONFIG=:4096:8
```

On a CPU-only machine use `--device cpu` in both commands; timings and floating-point trajectories can differ. The baseline defaults to the documented lagged EMA with decay 0.9. Additional CartPole seeds use `--seed 3818` and `--seed 3819` in separate output directories.

The Part 2 checkpoint includes the policy, Adam state, baseline numerator/mass, action RNG, and completed history. Resume requires matching parameters. The Deepdish command, dependency queue, and monitoring paths are in DEEPDISH.md. Both reference experiments now have their required Pong figures and statistics. They remain reproducible evidence of limited learning, alongside the much stronger CartPole results. Any improved experiment must be labeled separately.

Budgeted Pong follow-up

A separate 29.8-minute single-GPU campaign screened lower learning rates and current-frame-plus-motion observations. The selected model used two convolution layers (16 and 32 filters, kernel 5, stride 2), a 64-unit hidden layer, Adam 0.001, and the same lagged EMA baseline. It trained for 4,480 episodes before the budget stop. The checkpoint was selected using training scores only, then frozen for 500 games on a new seed bank beginning at 1,900,000,000.

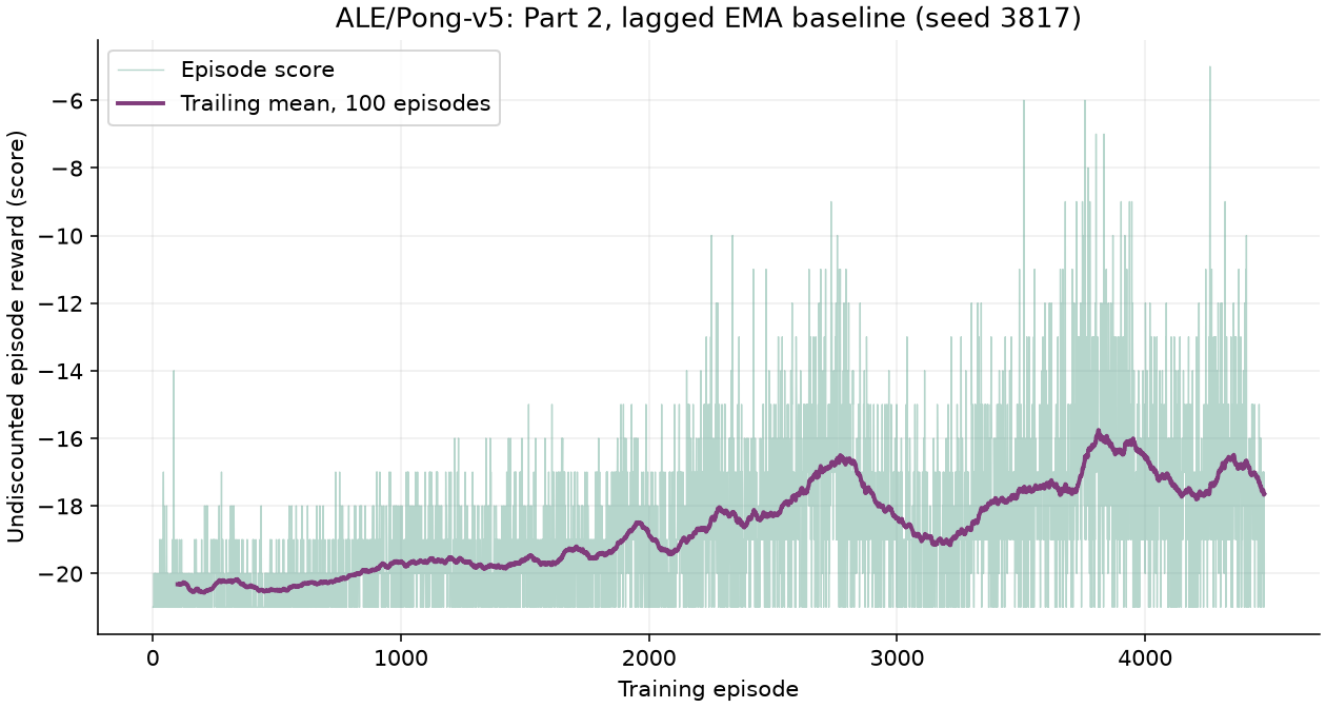


Figure 8. The budget-limited CNN follow-up. Its current-frame channel retains foreground even when frame differences are zero. The baseline and assigned discounted objective are unchanged.

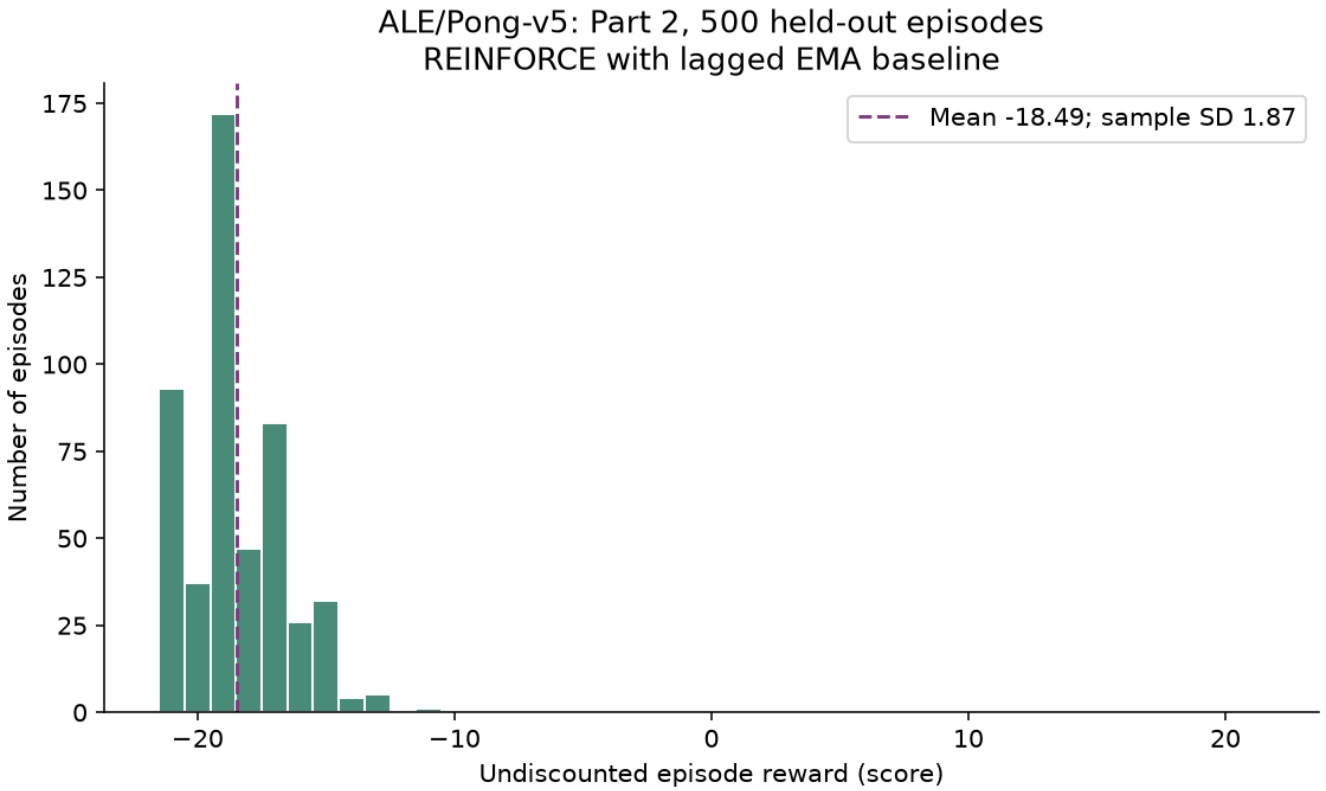


Figure 9. Mean score -18.492, sample SD 1.8722, approximate 95% interval [-18.6561, -18.3279], range -21 to -11. All games ended naturally, with no truncations, and **0/500 were wins**. This improves scoring over the Part 2 reference's -19.460 but does not resolve the winning-performance failure. Architecture, observations, learning rate, and budget differ from the reference; this is a tuning follow-up, not an isolated test of the baseline's effect.

The complete screening table, selection rule, validation, source snapshots, checkpoint, and remaining limitations are documented in PONG_TUNING.md and results/pong_tuning. Smaller rates alone did not improve the short difference-only pilots. New observations helped early training, but recent training scores were optimistic relative to the final frozen evaluation. No successful Pong agent or robust multi-seed improvement is claimed. GPU work finished within the campaign budget; no further experiment was queued.

With three additional months

Month 1: stronger variance estimates and optimization comparisons. Repeat matched experiments with more independent initialization/action streams and disjoint reset banks. Measure gradients at multiple checkpoints throughout training, with larger calibration and probe sets. Compare EMA decays and the gradient-norm-weighted optimal constant using training-only choices. Quantify baseline-estimation uncertainty as well as conditional probe uncertainty.

Month 2: richer baselines and controlled failure cases. Compare the scalar baseline with time-dependent and learned state-value baselines, fitted on past or cross-fitted data to avoid using an episode to fit its own baseline. Study stale baselines after demand/reward changes, negative and all-zero rewards, and rare early failures. Preserve the official game settings in required runs; label altered discounts and deterministic variants as diagnostic experiments.

Month 3: performance, robustness, and efficiency. Repeat held-out evaluation across training seeds and inspect learning deterioration, Pong win fractions, truncations, action entropy, and game lengths. Compare gradient variance, episode-score variance, and final score separately. Benchmark full update time, CPU use, GPU utilization, and I/O with confidence intervals, rather than only collector FPS. Check resumed versus uninterrupted GPU runs and validate any checkpoint-selection rule on a separate set before the final 500-game test.

Supplement: Single-GPU PPO Performance Study

The required experiments above use the requested basic policy-gradient algorithm. This additional study tests PPO-Clip with a learned state-value baseline, based on the original PPO paper and OpenAI's explanation. It is a different algorithm and is not substituted for the required comparison. Pong uses engineered image features rather than directly feeding binary frames to the neural network; object-centric Atari research motivates this representation. No emulator RAM, downloaded trained model, scripted action, shaped reward, extra frame skip, or Pong point reset is used.

Single-GPU execution and method

Fresh training and frozen evaluation both ran on one NVIDIA RTX PRO 5000 Blackwell GPU on Deepdish. Both trials selected the same physical GPU using CUDA_VISIBLE_DEVICES. The runner rejects anything other than exactly one visible GPU. Saved metadata verifies that all actor/value parameters and the optimizer's moment tensors are on CUDA; evaluation also records CUDA parameters. CPU workers simulate the official environments and prepare observations for the GPU.

Stable Baselines3 2.9.2 PPO uses separate actor and value networks, each with two 64-unit tanh layers, Adam 0.0003, 128-step rollouts per environment, minibatches of 256, GAE lambda 0.95, clip range 0.2, value coefficient 0.5, and gradient-norm limit 0.5. CartPole uses eight workers, ten optimization epochs and no entropy bonus. Pong uses sixteen workers, four epochs and entropy coefficient 0.01. The assigned discounts remain 0.95 and 0.99. Official environment versions, CartPole's 500-step limit, and Pong actions 2/3, frame skip 4 and sticky probability 0.25 are preserved. The SB3 documentation describes the maintained PPO implementation and its normalized advantages.

Pong's 12 inputs are image-derived ball and paddle positions, motion, relative positions, previous ball height and a visibility flag. Missing balls retain the last position with a missing flag; reset clears motion. These features do not prescribe the action. The feature validator checks synthetic centroids and motion, missing objects, unchanged input frames, and resets. Four actual games (4,135 transitions) exactly match official rewards and endings under matched actions. The kinematic-input option was validated but not used in these trials.

Frozen-policy results

Checkpoint selection uses only the best trailing mean over 100 completed training episodes. The selected CartPole checkpoint was saved after observing 258,128 steps and 936 completed episodes; Pong after 2,317,808 steps and 356 episodes. Counts can include an in-progress rollout; optimizer updates use completed rollout batches. Screening continues after selection, as shown in the plots. No final test games are used to choose checkpoints.

GPU-trained stochastic policy	Games	Mean score	Sample SD	Outcome
CartPole PPO	500	500.000	0.0000	500/500 at the cap
Pong PPO	500	-6.626	4.9727	45/500 positive-score games

Each test requests exactly 500 distinct seeds beginning at 1,960,000,000, disjoint from training. All requested seeds must finish; fast extra autoreset games are ignored rather than substituted for slow games. Parameters stay frozen and actions are sampled stochastically. The source snapshot, evaluated model SHA-256, CUDA provenance, seed bank, training-only selection and raw score statistics pass the independent saved-artifact audit.

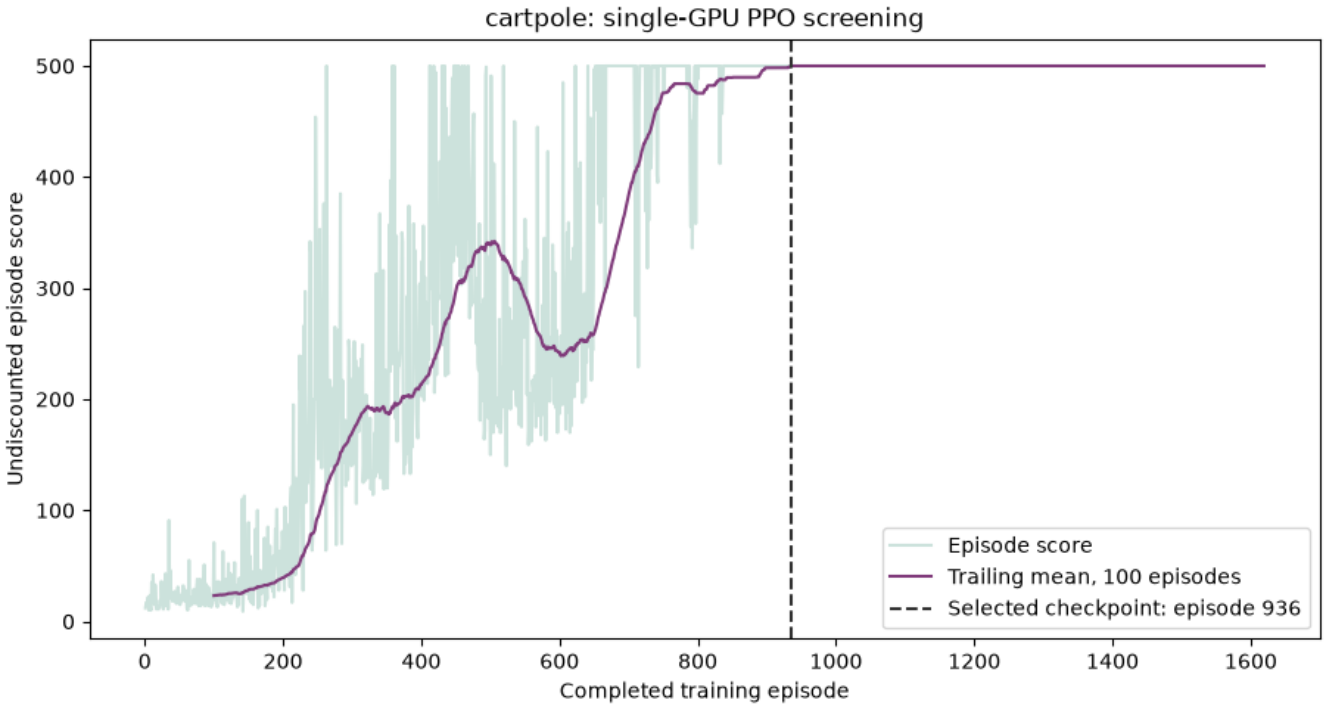


Figure S1. Raw training scores and exact trailing 100-episode mean. The vertical line marks the selected checkpoint; later screening scores are retained.

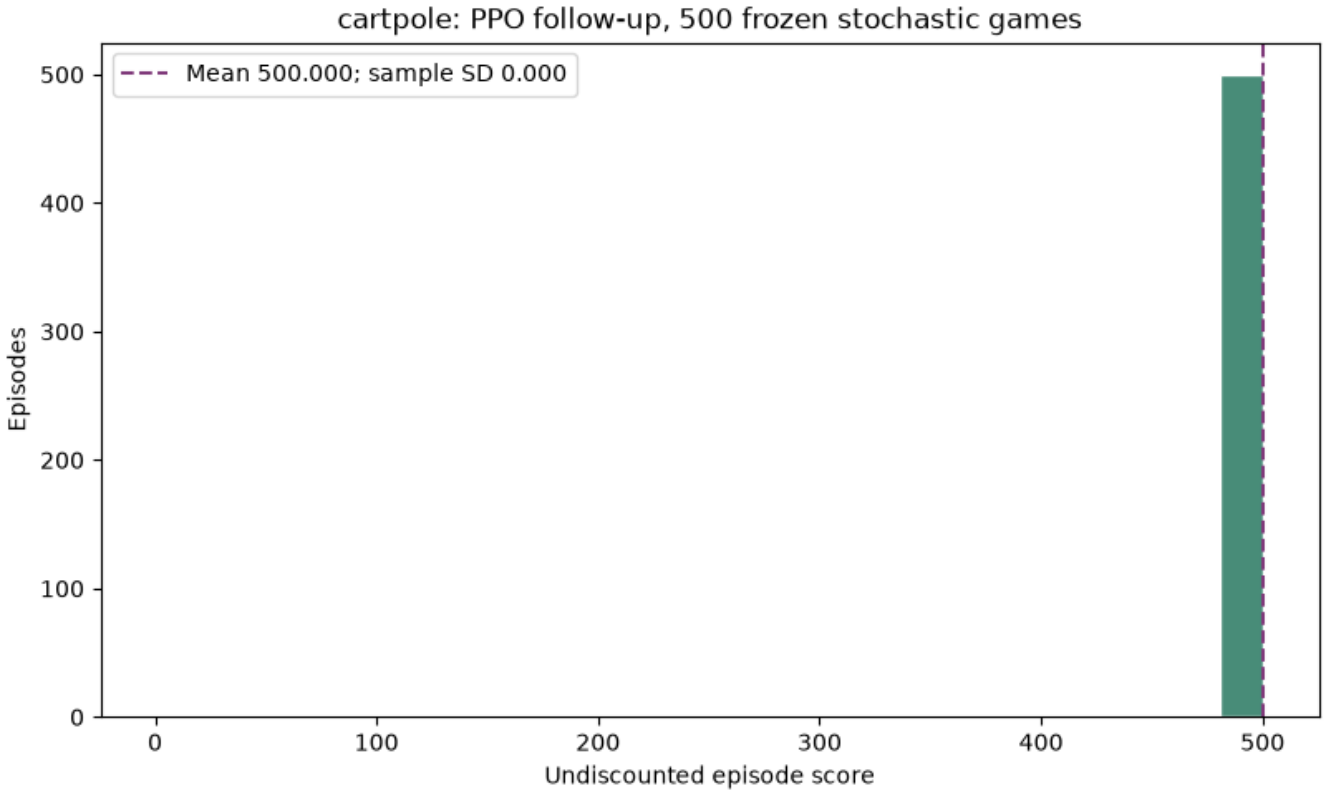


Figure S2. Exactly 500 frozen stochastic games. Reaching 500 is the official time-limit cap, not an observed physical failure. Zero empirical SD, if observed, does not imply zero population uncertainty.

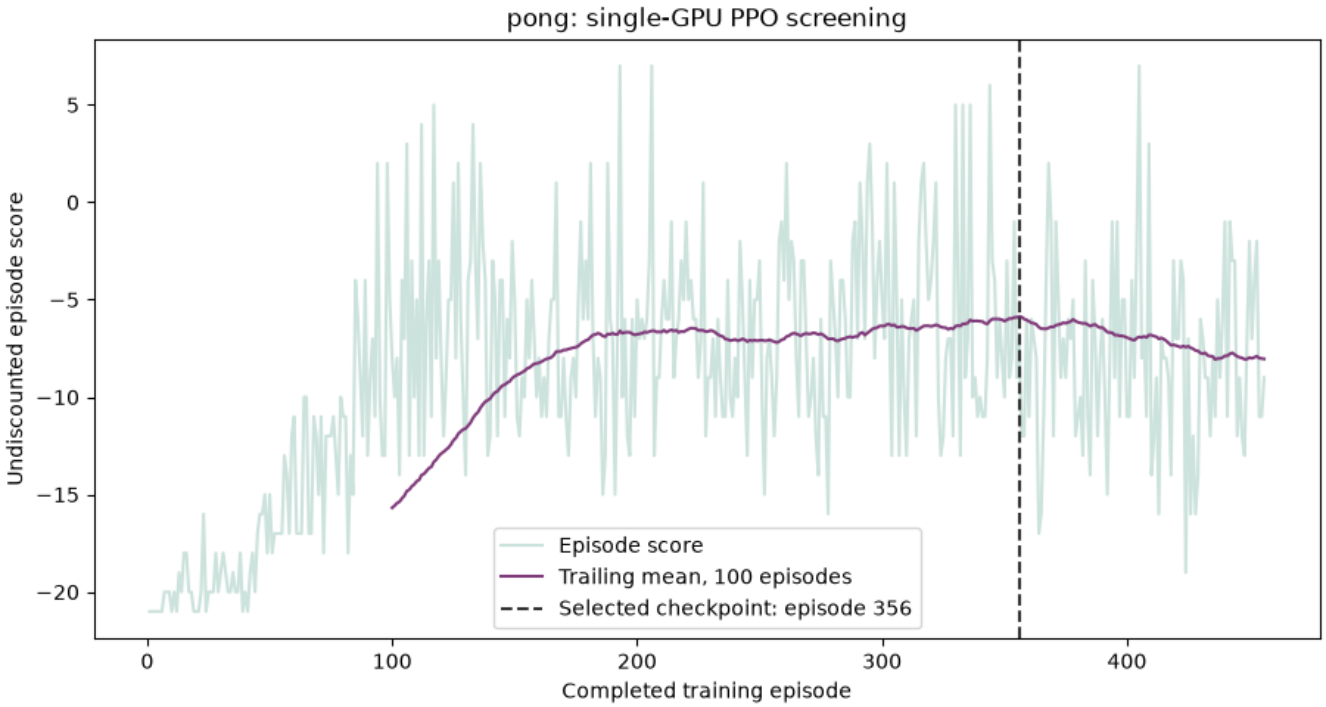


Figure S3. Raw full-game scores and trailing mean. A training window mixes changing policies and is not an estimate of one frozen checkpoint's performance.

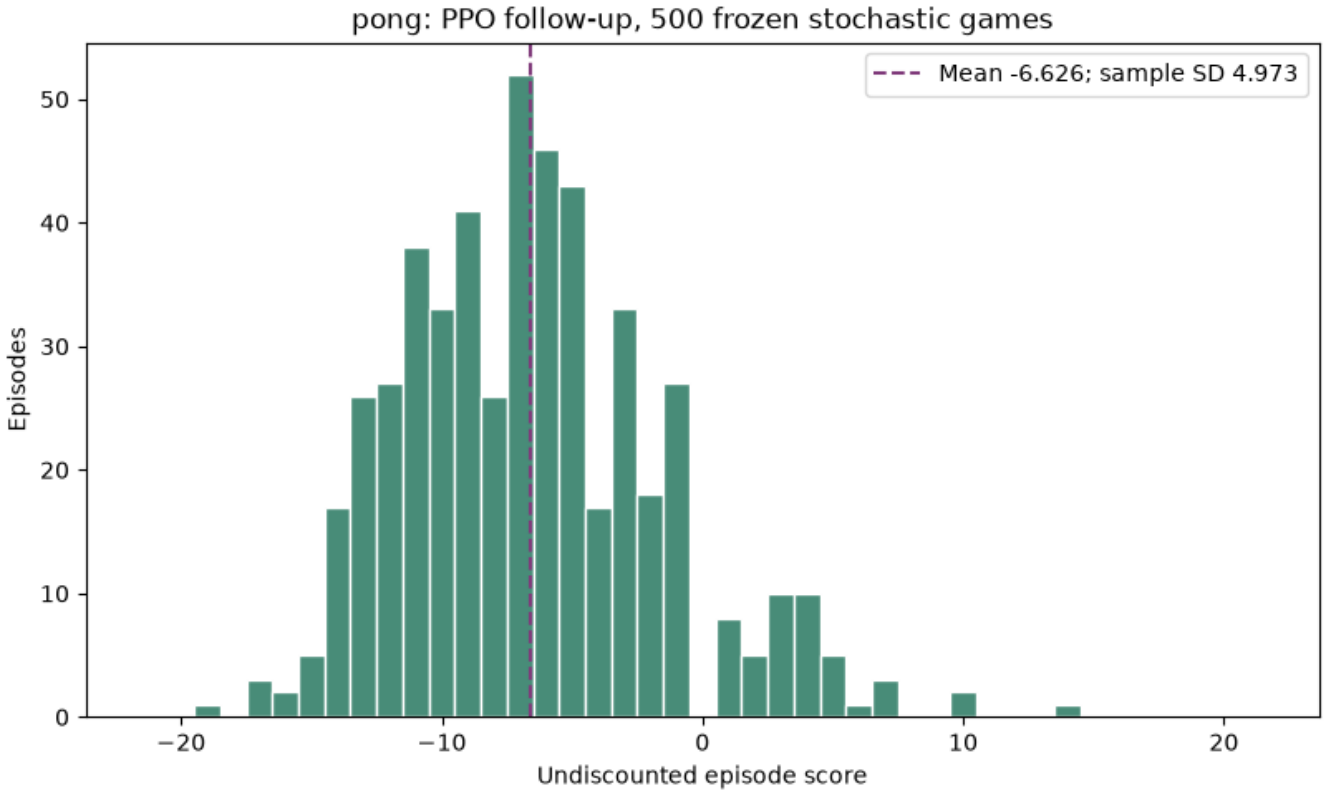


Figure S4. Exactly 500 reserved full games. Natural positive-score endings, truncations, and raw scores are retained separately in the audit and episode CSV.

Interpretation and reproduction

This study changes the baseline, estimator, optimization, network and Pong representation. Score differences cannot be attributed to the baseline alone. The results are conditional on one training seed, with different budgets and evaluation banks from the original experiments. Pong should not be described as solved based on a small number of wins. For CartPole, if all 500 games reach the cap, the exact one-sided 95% lower bound on its cap probability is 0.9940 under independent trials; the observed zero SD does not prove universal reliability.

Install requirements-ppo.txt in a separate environment with CUDA-enabled PyTorch. Expose your one assigned GPU and set CUBLAS_WORKSPACE_CONFIG=:4096:8. For a fresh run, replace both ISO timestamps below with future UTC deadlines, reserving time for all 500 final games:

```
python ppo_gpu.py --game cartpole --device cuda --workers 8 --steps 600000 \
  --train-until "YYYY-MM-DDTHH:MM:SS+00:00" \
  --finish-until "YYYY-MM-DDTHH:MM:SS+00:00" \
  --output results/ppo_gpu/reproduction_cartpole
```

Use –game pong –workers 16 and a fresh output directory for Pong. Run `python validate_ppo_gpu.py` for feature/environment validation and `python audit_ppo_gpu.py` to recompute the saved statistics, CUDA/source/model checks, training plots and this supplement. The full environment lock, plans, source snapshots, selected models and raw data are under results/ppo_gpu.